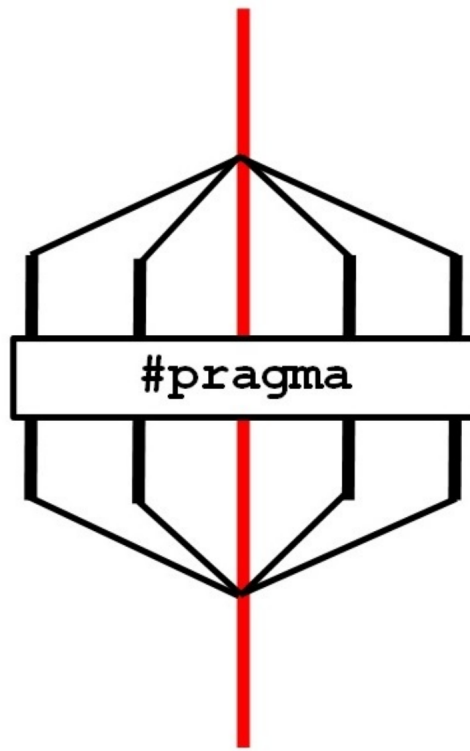
Валентин Юльевич Арьков

Организация параллельных потоков. Часть 2

Учебное пособие



Валентин Юльевич Арьков

Организация параллельных потоков. Часть 2

Учебное пособие

Издательские решения
По лицензии Ridero
2020

УДК 004
ББК 32.973
А89

Шрифты предоставлены компанией «ПараТайп»

Арьков Валентин Юльевич

А89 Организация параллельных потоков. Часть 2 : Учебное пособие / Валентин Юльевич Арьков. — [б. м.] : Издательские решения, 2020. — 124 с.
ISBN 978-5-4498-5445-2 (т. 2)
ISBN 978-5-4498-5446-9

При решении инженерных, экономических и научных задач используются высокопроизводительные вычисления — High Performance Computing или сокращённо HPC.

Параллельные программы нужны для того, чтобы использовать вычислительные мощности многоядерных процессоров и графических ускорителей. В данной работе мы рассмотрим технологию автоматической организации параллельных потоков для многоядерных вычислительных машин.

УДК 004
ББК 32.973

12+ В соответствии с ФЗ от 29.12.2010 №436-ФЗ

ISBN 978-5-4498-5445-2
ISBN 978-5-4498-5446-9

© Валентин Юльевич Арьков, 2020

ОГЛАВЛЕНИЕ

Введение 5
1. Общие сведения 6
1.1. Информация и литература 6
1.2. Оформление отчёта 7
2. Технология OpenMP 9
2.1. Ключевые понятия 9
2.2. Многопоточность 10
2.3. Термины OpenMP 11
2.4. Ответственность разработчика 12
2.5. Инструменты OpenMP 13
3. Составление параллельных программ 16
3.1. Hello, World! 16
3.2. Поддержка OpenMP 20
3.3. Первая параллельная программа 24
3.4. Параллельные и последовательные области 28
3.5. Число параллельных потоков 29
3.5.1. Переменная окружения 29
3.5.2. Библиотечная функция 35
3.5.3. Параметр директивы 37
4. Управление распараллеливанием 40
4.1. Вычислительная среда 40
4.1.1. Число процессоров 40
4.1.2. Число потоков 42
4.2. Параллельный цикл 44
4.2.1. Ситуация гонки 44
4.2.2. Редукция 51
4.2.3. Порядок выполнения потоков 54
4.2.4. Локальные копии переменной 61
4.3. Вычислительная нагрузка 65
4.3.1. Полная загрузка 65
4.3.2. Неполная загрузка 67
4.3.3. Привязка к ядрам 68
4.4. Время вычислений 72

5. Параметры командной строки 75
5.1. Команда DIR 76
5.2. Чтение параметров 77
5.3. Использование параметров 82
5.4. Управление циклом 86
5.5. Анализ эффективности 88
5.5.1. Пакетный файл 89
5.5.2. Сводная таблица 92
5.5.3. Организация экспериментов 100
6. Численное интегрирование 106
6.1. Варианты заданий 106
6.2. Аналитическое решение 107
6.3. Последовательная программа 108
6.3.1. Одинарная точность 108
6.3.2. Двойная точность 115
6.4. Параллельные потоки 115
Заключение 120
Литература 121

ВВЕДЕНИЕ

Высокопроизводительные вычисления стали сегодня реальностью не только на уровне суперкомпьютеров и вычислительных кластеров, но и для персональных компьютеров и мобильных устройств. Речь идёт о решении инженерных, экономических и научных задач с использованием высокопроизводительных вычислений и написанием параллельных приложений (прикладных программ).

Параллельные программы нужны для того, чтобы использовать вычислительные мощности многоядерных процессоров и графических ускорителей. В данной работе мы рассмотрим технологию автоматической организации параллельных потоков для многоядерных вычислительных машин.

Нам предстоит оценить параметры эффективности распараллеливания программы на конкретной конфигурации вычислительной системы. По результатам измерения быстродействия определяются показатели ускорения и эффективности распараллеливания. Это опыт исследования и практического знакомства с технологией — не по учебнику, а в форме личногго знакомства. Каждое положение и утверждение можно проверить, «покрутить в руках» и убедиться в его правильнсти или ошибочности.

Мы рассмотрим задачу численной оценки значения определённого интеграла — по двум причинам. Во-первых, многие практические задачи сводятся к нахождению суммы или интеграла. Во-вторых, численные методы интегрирования хорошо поддаются распараллеливанию. Каждое отдельное значение подынтегрального выражения можно вычислять независимо от всех остальных значений. Поэтому численное интегрирование — подходящая задача для знакомства с высокопроизводительными вычислениями.

1. ОБЩИЕ СВЕДЕНИЯ

1.1. ИНФОРМАЦИЯ И ЛИТЕРАТУРА

Параллельное программирование освещается в большом количестве учебников и пособий [1–9].

При изучении параллельного программирования полезно обращаться к библиотеке учебных материалов Лаборатории параллельных информационных технологий НИВЦ МГУ. Доступ к библиотеке осуществляется по адресу:

http://parallel.ru/info/parallel/

Учебник и учебные пособия, представленные на указанном сайте, предназначены для использования студентами вузов и доступны для бесплатного скачивания.

В данной работе мы будем опираться на некоторые примеры из учебного пособия А. С. Антонова [8]. Для первого знакомства с технологиями мы разбираем каждый пример достаточно подробно. Попутно мы обсуждаем самые общие вопросы.

Всё это нужно, чтобы студент не просто освоил стандартные, шаблонные действия с конкретным программным продуктом. В любом деле нужны специалисты с кругозором и эрудицией, с пониманием и способностью самостоятельно развиваться. А это требует чего-то большего, чем только узкопрофессиональные знания и конкретные умения.

На сегодняшний день в интернете имеется множество онлайн курсов.

Первый пример — Национальный Открытый Университет ИНТУИТ:

https://www.intuit.ru.[1]

Основной ресурс с отечественными массовыми открытыми

онлайн-курсами (МООК) — «Открытое образование»:

https://openedu.ru/.

Международная платформа МООК «Курсера»:

https://www.coursera.org/.

Задание. Найдите на перечисленных сайтах курсы по следующим ключевым словам и перечислите их в отчёте:

— параллельные;
— parallel;
— высокопроизводительные;
— high performance computing;
— суперкомпьютеры;
— supercomputer;
— OpenMP;
— HPC;
— многоядерные;
— multicore.

1.2. ОФОРМЛЕНИЕ ОТЧЁТА

Отчёт по работе оформляем точно так же, как и в предыдущих работах [10]. Отчёт делаем в виде рабочей книги Excel. Это многостраничная книга с оглавлением.

Вначале, как и положено, должен быть титульный лист со всеми данными о работе и исполнителе.

Затем идёт оглавление со ссылками на все страницы.

Далее — задание.

Следом — шаги выполнения работы.

Текст программы вставляем как текст, а не как картинку.

На каждом листе — заголовок и пояснения о том, что заложено в данной программе. Что она должна делать и как это реализовано. Здесь же копия экрана и пояснения по поводу результатов работы.

[1] https://www.intuit.ru/

Поскольку листов в отчёте будет много, названия листов (на вкладках) содержат только номера страниц. Подробные названия нужны в верхней части листа и в оглавлении.

Задание. Создайте файл отчёта и заполните титульный лист.

2. ТЕХНОЛОГИЯ OPENMP

В данной работе мы знакомимся с технологией автоматического распараллеливания программ **OpenMP**.

Название расшифровывается следующим образом:

Open Multi-Processing.

Распараллеливание программ поддерживается для двух языков программирования

— Fortran;

— C/C++.

2.1. КЛЮЧЕВЫЕ ПОНЯТИЯ

В результате использования данной технологии компилятор автоматически генерирует многопоточные программы. Такие программы дают эффект ускорения при запуске на многоядерных системах с общей памятью.

Многоядерные компьютеры — это так называемые «системы с общей памятью». Другое название — разделяемая память. Английское название: SHARED MEMORY.

Имеется в виду совместное использование оперативной памяти: любой поток имеет доступ к общим глобальным переменным процесса. Более красивая официальная формулировка звучит так: «общий доступ параллельных потоков к виртуальному адресному пространству текущего процесса».

Задание. Изучите в Википедии следующие статьи и выясните, что означают эти термины:

— «Multiprocessing» или «Многопроцессорность»;

— «Multithreading (computer architecture)» или «Многопоточность»;

— «Многоядерный процессор» или «Multi-core processor»;

— «OpenMP».

Задание. Изучите историю разработки технологии и версии спецификаций для обоих языков программирования:

https://www.openmp.org/specifications/.

OpenMP использует наиболее распространённую модель параллелизма:

Single Instruction Multiple Data (SIMD).

В этом случае параллельная часть программы состоит из нескольких одинаковых потоков, которые обрабатывают разные наборы данных.

Задание. Изучите в Википедии статьи «SIMD» и «Таксономия Флинна». Выясните, что такое SIMD.

2.2. МНОГОПОТОЧНОСТЬ

Использование OpenMP проще всего изучать на компьютере с одним многоядерным процессором.

Чтобы сделать из последовательной программы параллельную, разработчику нужно добавить всего несколько строк. Компилятор может игнорировать директивы распараллеливания. В этом случае мы получаем исходную последовательную программу. Если настроить компилятор на использование OpenMP, он автоматически сгенерирует и скомпилирует параллельную программу.

В технологии OpenMP происходит динамическое создание потоков. В процессе работы программа может создавать и уничтожать дополнительные потоки (рис. 2.1). Такая модель условно называется «fork-join». Это означаетразделение программына несколько параллельных веток, выполнение работы параллельном режиме и последующее слияние в одну последовательную ветку.

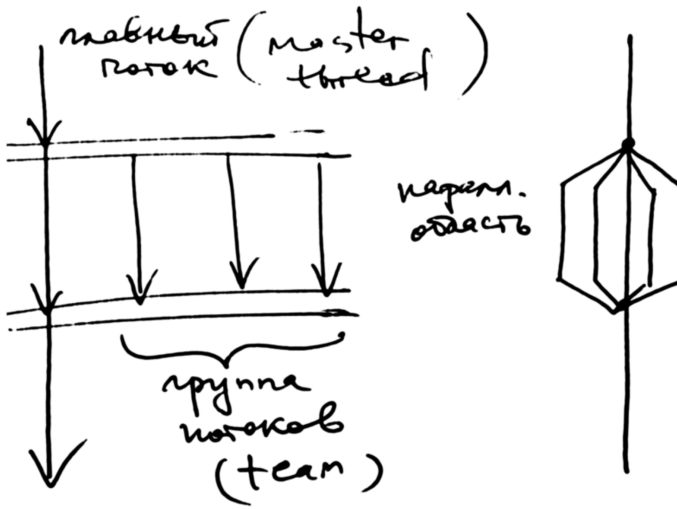


Рис. 2.1. Модель распараллеливания fork-join

Задание. Изучите в Википедии статью «Fork join».

2.3. ТЕРМИНЫ OPENMP

В технологии OpenMP используют следующие понятия и термины.

Construct — **Конструкция** — директива и следующий за ней структурный блок команд (часть программы, ограниченная фигурными скобками).

Directive — **Директива** — строка, начинающаяся как **#pragma omp** и определяющая поведение программы.

Thread — **Поток** — часть программы, которая может выполняться последовательно на одном ядре.

Master thread — Главный поток — поток, который существует на протяжении всего времени выполнения процесса и который создаёт группу параллельных потоков (**Team**) при входе в парал-

лельную область.

Team — Группа параллельных потоков — один или несколько потоков, которые участвуют в выполнении какой-либо конструкции языка.

Параллельная программа состоит из параллельных и последовательных областей.

Parallel region — Параллельная область — часть программы, которая может выполняться несколькими потоками.

Serial region — Последовательная область — часть программы, которая выполняется только главным потоком.

Переменные делятся на два вида — в зависимости от способа доступа:

Private — уникальная переменная, доступная только внутри потока.

Shared — общая переменная, доступная всем параллельным потокам в рамках одной группы.

2.4. ОТВЕТСТВЕННОСТЬ РАЗРАБОТЧИКА

Лёгкость распараллеливания оборачивается повышением ответственности программиста.

Конкретная реализация OpenMP (среда разработки, компилятор и библиотека) не обязательно проверяет корректность распараллеливания и возникновение следующих нежелательных ситуаций:

— **Dependencies** — зависимости по данным между параллельными потоками;

— **Conflicts** — конфликты доступа к данным;

— **Deadlocks** — тупиковые ситуации (взаимная блокировка);

— **Race conditions** — ситуация гонки за доступ к данным.

Как видим, все эти ситуации связаны с обращением к общим данным из нескольких параллельных потоков. В последовательных программах таким проблем просто не может возникнуть.

Ответственность за корректность составления программы лежит полностью на составителе программы. Все эти положения можно найти в тексте спецификации OpenMP.

2.5. ИНСТРУМЕНТЫ OPENMP

Для реализации параллелизма в технологии OpenMP используют три вида инструментов:

— **Directives** — Директивы компилятора;

— **Library functions** — Готовые библиотечные функции;

— **Environment variables** — Переменные среды (параметры окружения).

2.5. Среда разработки

Всё программное обеспечение, используемое в данной работе, является бесплатным и доступно на официальных сайтах фирм-разработчиков. Программы устанавливают в операционной системе **Microsoft Windows**.

Интегрированная среда разработки **Microsoft Visual Studio Community Edition** предоставляется бесплатно для студентов, индивидуально работающих программистов и разработчиков программного обеспечения с открытым исходным кодом **Open Source.**

В данной работе мы будем использовать **Visual Studio**. Не потому, что это самый лучший компилятор. И не потому, что мы хотели бы заработать на рекламе конкретного программного продукта. Просто под руку попалось. Работает для наших задач — и на том спасибо.

Желающие могут работать в любой другой операционной системе и использовать любой другой компилятор языка программирования Си. Всё, что требуется — это поддержка технологии распараллеливания OpenMP.

Устанавливаем **Visual Studio.**

Переходим на сайт:

https://visualstudio.microsoft.com/.

Выбираем версию **Community Edition** (рис. 2.2). Она бесплатна для учебных и некоммерческих целей.

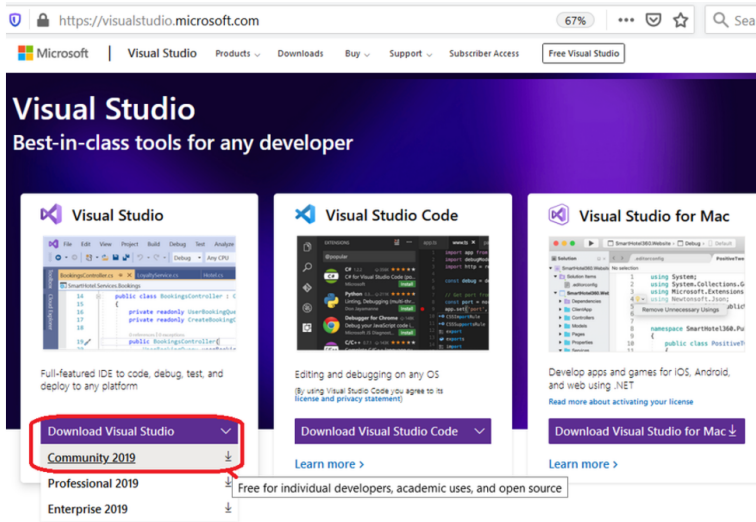


Рис. 2.2. Выбор версии среды разработки

Раньше, в далёком прошлом (несколько лет тому назад) программы были небольшие, и разработчики предлагали скачать образ диска в формате ***.ISO**. Его можно было даже на «болванку» записать. На записываемый **DVD-R** или на многоразовый, перезаписываемый **DVD-RW**.

Сегодня выбора почти не осталось. Есть только web-установщик. Небольшая программа, которая скачает из интернета необходимые компоненты для выбранной конфигурации. Счёт может идти на десятки гигабайт, а то и поболее.

Нажимаем кнопку «Установить» и начинается скачивание установщика (рис. 2.3).

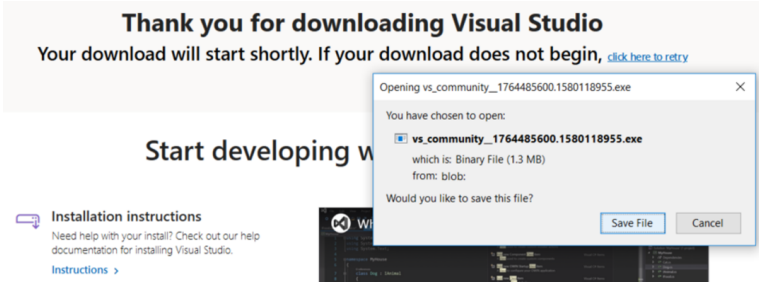


Рис. 2.3. Скачивание веб-установщика

Как видим, пока мы скачали чуть больше одного мегабайта.

Скачанную программу запускаем. Начинается процесс установки. В дополнение к стандартному набору средств разработки, который нам предлагают по умолчанию, мы выбираем компилятор языка Си. Остальные компоненты тоже можно установить, если будет такое желание ии необходимость. Веб-установщик скачает всё, что потребуется для завершения установки.

Задание. Установите среду разработки и компилятор языка Си.

3. СОСТАВЛЕНИЕ ПАРАЛЛЕЛЬНЫХ ПРОГРАММ

В этом разделе мы составим несколько программ, постепенно усложняя алгоритмы.

Каждую программу мы исследуем и рассмотрим со всех сторон.

Попутно мы уточним и проясним разные моменты и особенности.

3.1. HELLO, WORLD!

Убедимся, что среда разработки позволит нам сделать что-нибудь полезное. Мы начинаем работу с нуля. С самой простой программы «Всем привет!». Английское название: «Hello, World!»

Создаём новый проект. Пустой, незаполненный проект.

Добавляем новый элемент проекта — файл с исходным текстом программы (рис. 3.1).

Компилируем программу и запускаем её на выполнение.

Для каждой новой программы создаём новый каталог и новый файл с исходным текстом программы. Иначе потом следов не найдёшь. В руках останется последняя недоделанная программа, которая затёрла всю предыдущую работу.

E:\Arkov-OMP\Hello-1\Hello-1.c

```
1  #include <stdio.h>
2
3  void main()
4  {
5      printf("Hello World!\n");
6  }
7
```

Рис. 3.1. Программа «Всем привет»

Устанавливаем конфигурацию **Release**.

Нажимаем комбинацию клавиш:

Ctrl + F5.

Видим результаты работы программы в командном окне.

Задание. Создайте программу «Всем привет!» и запустите её на выполнение.

В данной работе мы будем иметь дело только с вариантом **Release** и не будем заниматься отладкой программы в режиме **Debug**. Чтобы больше не отвлекаться на эти красивые названия, их нужно будет разобрать и изучить.

Задание. Изучите в Википедии статьи «Отладка программы», «Debugging», «Software bug», «Стадии разработки программного обеспечения» и «Software release life cycle». Выясните значение, происхождение и перевод терминов **DEBUG** и **RELEASE**.

По умолчанию шрифт командного окна довольно мелкий.

Настроим его покрупнее.

Щелкаем по значку в левом верхнем углу окна (рис. 3.2) и выбираем свойства:

Properties.

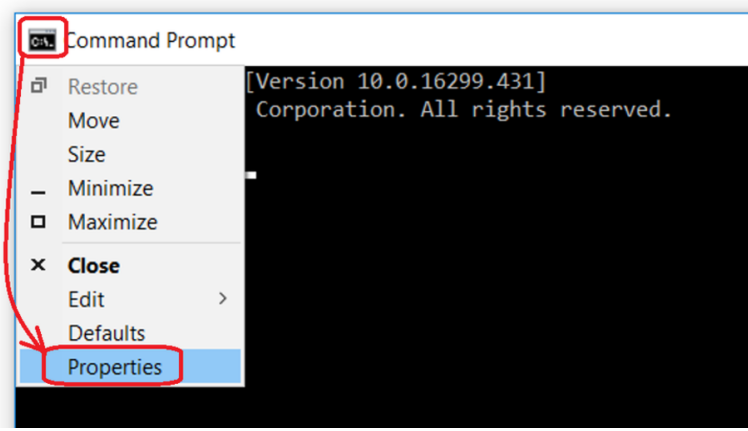


Рис. 3.2. Свойства командного окна

Здесь нас интересует раздел шрифтов:

Font.

Выбираем любой моноширинный шрифт (рис. 3.3). Это означает, что все буквы одинаковой ширины. В названии некоторых шрифтов может быть слово **mono**. «Моно» означает «один». Эти шрифты имитируют первые печатные устройства, похожие на печатные машинки. Моноширинный шрифт позволяет расположить результаты вывода на экран ровными столбцами.

Задаём подходящий размер.

Если буквы плохо различимы, делаем шрифт жирным.

В нашем примере мы выбрали такой шрифт:

— Size — 24;

— Font — Courier New;

— Bold fonts.

В нижней части окна выбора шрифта выводится пример текста на экране и указаны размеры букв в пикселах (точках).

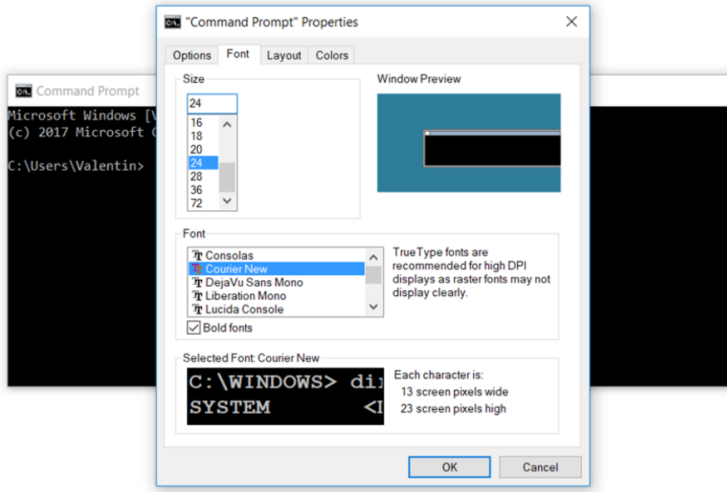


Рис. 3.3. Выбор шрифта командного окна

Задание. Настройте шрифт командного окна.

После установки шрифта ещё раз запускаем программу на выполнение.

Теперь буковки стали более читабельными (рис. 3.4).

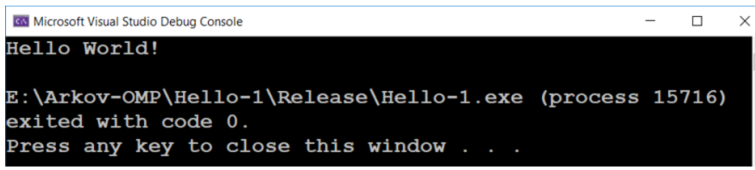


Рис. 3.4. Результаты работы программы

Задание. Убедитесь, что настроили шрифт командного окна.

3.2. ПОДДЕРЖКА OPENMP

Проверим, как есть ли в нашем компиляторе поддержка распараллеливания.

Если в компиляторе включена поддержка OpenMP, то появится следующий идентификатор:

_OPENMP.

Для проверки задания макроса используется директива IF DEFINED:

#ifdef.

Директива предлагает компилятору проверить, определён ли указанный идентификатор.

Запускаем следующую программу (рис. 3.5).

E:\Arkov-OMP\omp-01-check\omp-01-check.c

```
 1  #include <stdio.h>
 2
 3  void main()
 4  {
 5  #ifdef _OPENMP
 6      printf("OpenMP Date: %d\n", _OPENMP);
 7  #else
 8      printf("Sorry, No OpenMP\n");
 9  #endif
10  }
```

Рис. 3.5. Проверка поддержки OpenMP

Запущенная программа сообщает нам, что указанный макрос не определён (рис. 3.6).

Это означает, что после компиляции мы получаем самую обычную последовательную программу.

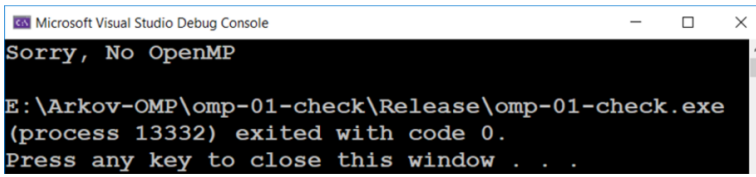


Рис. 3.6. Поддержки OpenMP пока нет

Задание. Составьте и запустите программу (рис.3.5).

Теперь включим поддержку распараллеливания. Настроим свойства текущего проекта:

Project — Properties (рис. 3.7).

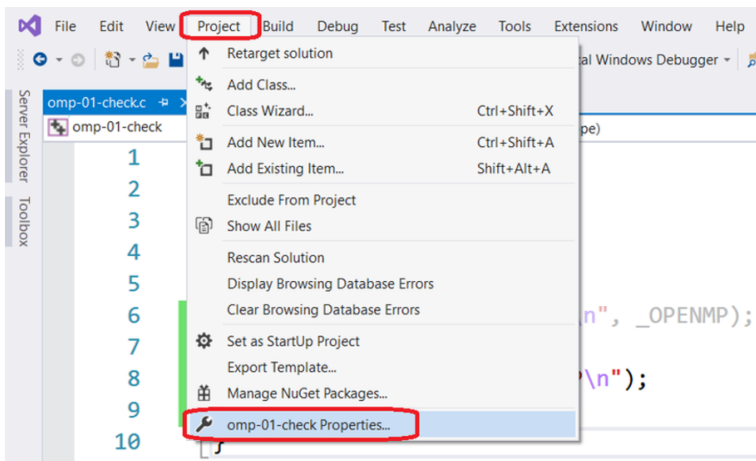


Рис. 3.7. Свойства проекта

Делаем следующие настройки (рис. 3.8):

Configuration — All Configurations;

Configuration Properties — C/C++ Language — OpenMP

Support — Yes.

В нижней части окна нам сообщают, что поддерживается версия 2.0:

OpenMP Support — Enable OpenMP 2.0 language extensions.

Здесь же показана опция командной строки, которая используется при вызове компилятора:

/openmp.

Нажимаем клавиши:

Apply — OK.

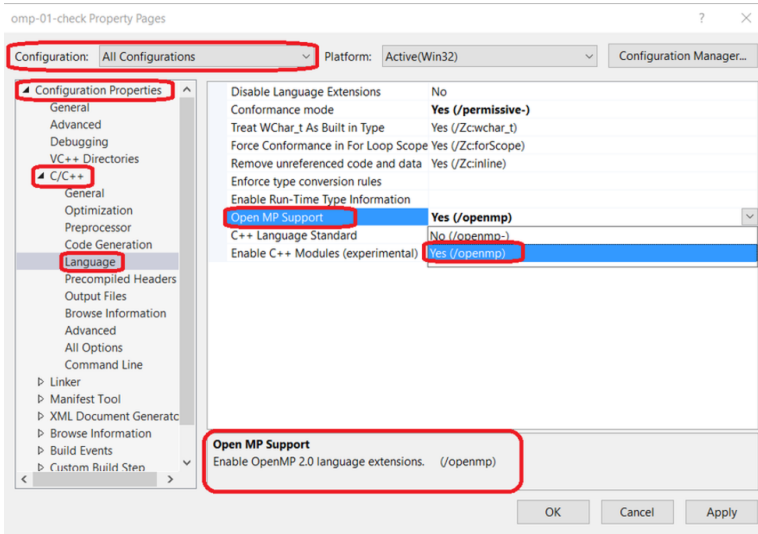


Рис. 3.8. Настройка свойств проекта

Задание. Включите поддержку OpenMP.

В окне редактирования программы сразу видно, что у нас включена поддержка OpenMP (рис. 3.9). Наводим курсор на идентификатор **_OPENMP** и видим, что он был определён.

Где-то ещё имеется следующая строчка:

#define _OPENMP 200203.

Это означает, что перед компиляцией программы все ссылки на данный идентификатор заменяются на **200203**.

Кроме того, теперь у нас подсвечивается ветка, которая фактически будет вставлена в текст программы.



Рис. 3.9. Окно редактора с поддержкой OpenMP

Задание. Изучите изменения в окне редактора.

Запускаем программу.

На экран выводится то самое загадочное число **200203** (рис. 3.10). Возможно, у вас будет что-то другое.

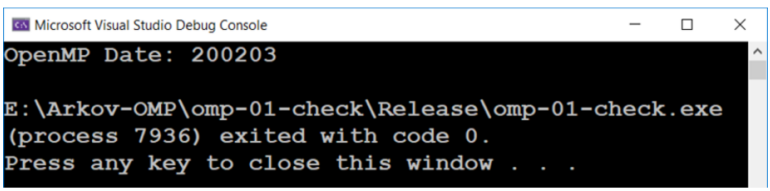


Рис. 3.10. Сообщение о поддержке OpenMP

Задание. Запустите программу на выполнение.

Полученное число сообщает нам год и месяц выпуска очередной версии спецификации.

Чтобы понять смысл сообщения, вернемся к списку версий: https://www.openmp.org/specifications/

Находим дату: март 2002 года (рис. 3.11).

Читаем, какая версия была выпущена тогда.

Для первого знакомства с технологией и для наших простых экспериментов этого более чем достаточно. Мы не используем сложные и продвинутые инструменты.



Рис. 3.11. Версии спецификаций

Задание. Найдите версию спецификации для языка Си по дате выпуска.

3.3. ПЕРВАЯ ПАРАЛЛЕЛЬНАЯ ПРОГРАММА

Компилятор работает и поддерживает распараллеливание. Составим самую первую параллельную программу.

Добавляем одну строчку (рис. 3.12):

#pragma omp parallel.

Мы объявляем часть программы, которая будет одновременно выполняться несколькими потоками.

E:\Arkov-OMP\omp-02-hello\omp-02-hello.c

```
1  #include <stdio.h>
2
3  void main()
4  {
5  #pragma omp parallel
6      printf("Hello World!\n");
7  }
```

Рис. 3.12. Параллельная программа

Запускаем программу и... ничего не изменилось (рис. 3.13). Это всё ещё последовательная программа «Всем привет!»

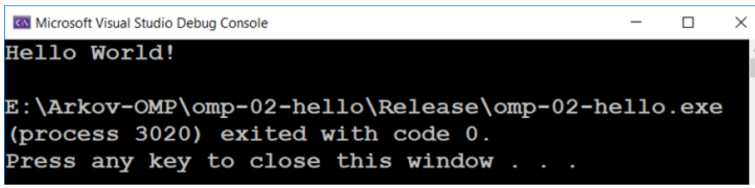


Рис. 3.13. Последовательная программа

Задание. Создайте и запустите программу (рис. 3.12).

Настроим поддержку OpenMP для текущего проекта.

Эту настройку придётся делать для каждого нашего проекта.

Запускаем программу и получаем восемь одинаковых сообщений (рис. 3.14).

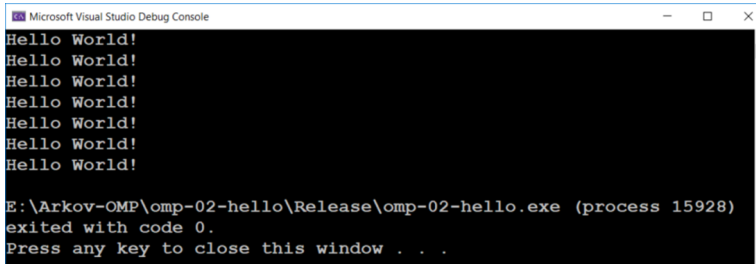


Рис. 3.14. Параллельное выполнение программы

Задание. Настройте проект и запустите параллельную программу.

Почему именно восемь?

Запустим **Диспетчер задач — Task Manager** (рис. 3.15). Для этого одновременно нажимаем три «волшебных» клавиши:

Ctrl + Alt + Del.

Кстати, как правильно произносится английское слово CONTROL? Большинство опрошенных студентов говорят с ударением на первом слоге. Потому что в английском обычно «ударяют» на первый слог. Но это правило работает не всегда. Особенно для тех слов, которые пришли из французского.

Задание. Выясните, как правильно произносится слово CONTROL.

Итак, мы запустили Диспетчер задач:

Task Manager.

Переходим на вкладку Быстродействие:

Performance.

Перед нами появляется некоторое количество окошек с графиками — по числу логических процессоров.

Если у нас всего один график, щёлкаем по нему правой кнопкой и выбираем в контекстном меню:

Change graph to — Logical processors.

Здесь нам намекают именно на «логические процессоры», а не на физические процессоры и не на ядра процессора. Для операционной системы и для параллельной программы эти различия и тонкости не слишком важны. В частности, на нашем компьютере один процессор, в котором имеются четыре ядра с «гипертредингом». **HyperThreading** — это технология компании Intel для выполнения двух потоков на одном физическом ядре. Итого имеем восемь виртуальных, логических процессоров.

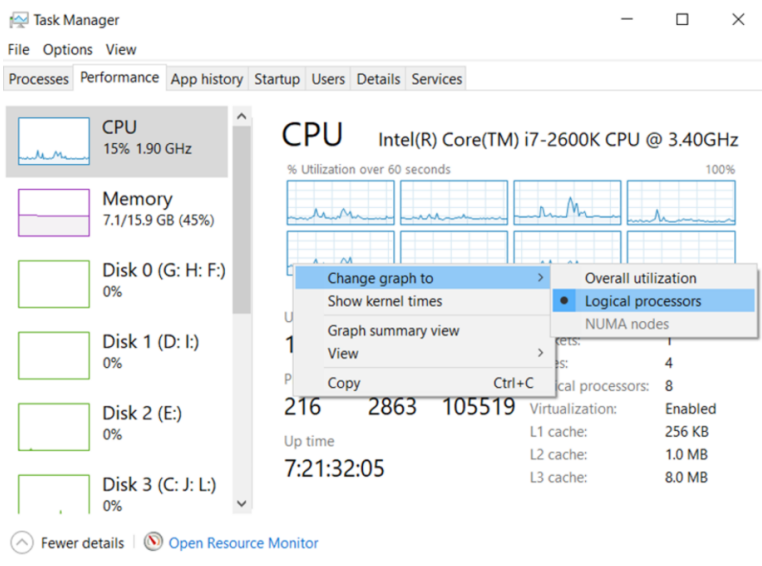


Рис. 3.15. Диспетчер задач

Задание. Выясните количество виртуальных процессоров на своём компьютере.

3.4. ПАРАЛЛЕЛЬНЫЕ И ПОСЛЕДОВАТЕЛЬНЫЕ ОБЛАСТИ

Как мы уже говорили, здесь используется модель «fork-join».

Составим программу, в которой организуем три области:

1) первая последовательная область печатает единичку;

2) параллельная область печатает двоечку — несколько раз, по количестве виртуальных процессоров;

3) вторая последовательная область печатает троечку.

Текст программы приводится на рис. 3.16.

E:\Arkov-OMP\omp-03-fork-join\omp-03-fork-join.c

```
 1  #include <stdio.h>
 2
 3  void main()
 4  {
 5  printf("1\n");
 6
 7  #pragma omp parallel
 8      printf("2\n");
 9
10  printf("3\n");
11  }
```

Рис. 3.16. Программа с тремя областями

Включаем в компиляторе поддержку OpenMP.

Компилируем.

Запускаем.

Рассматриваем результаты (рис. 3.17).

На экран выводится одна цифра **1**, восемь цифр **2** и одна цифра **3**.

Это говорит о том, что рспараллеливанию подвергается только часть программы. Та часть, которую мы указали в директиве **parallel**.

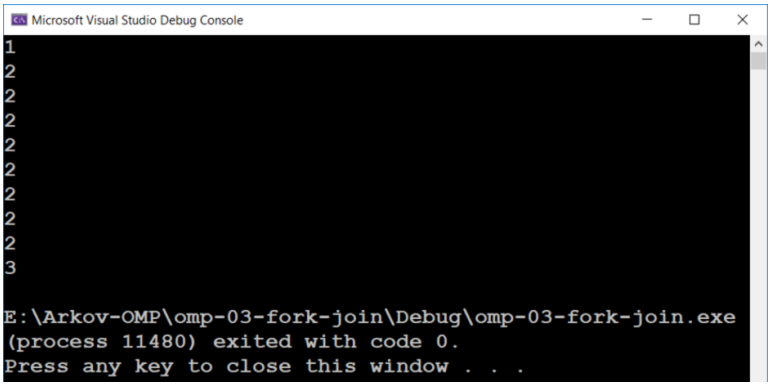


Рис. 3.17. Результаты работы параллельной программы

Задание. Запустите параллельную программу (рис. 3.16).

3.5. ЧИСЛО ПАРАЛЛЕЛЬНЫХ ПОТОКОВ

У нас имеется три способа задания числа параллельных потоков:

— параметр окружения **OMP_NUM_THREADS**;
— функция **omp_set_num_threads**;
— параметр директивы **num_threads**.

3.5.1. ПЕРЕМЕННАЯ ОКРУЖЕНИЯ

Начнём с задания часта потоков с помощью переменной окружения. Другие названия: параметр окружения, параметр среды, set-параметр.

Английское название:
Environment variable.

Это текстовая переменная операционной системы. Переменная находится в оперативной памяти. Её значение может прочитать любая программа в момент выполнения. К переменным окружения обращаются по имени (названию). Окружение, или среда, — это среда выполнения программ. Эту среду создаёт и обслуживает операционная система.

Мы будем работать с параметрами окружения через командное окно. Другие названия: командная строка, командный процессор, командная оболочка. Английские названия:

— **Command Window**
— **Command Processor**
— **Command Shell.**

Нажимаем комбинацию клавиш:

Win + R.

Вводим имя программы, которая обслуживаем командное окно:

CMD (рис. 3.18).

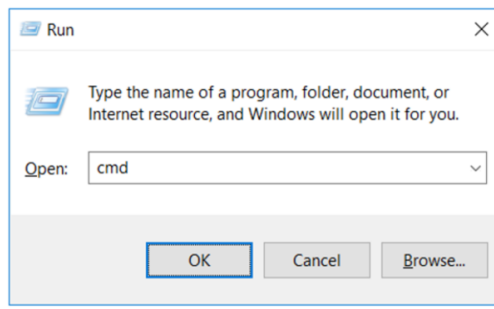


Рис. 3.18. Запуск командного окна

Задание. Запустите командное окно.

Вводим в командной строке команду просмотра параметров окружения:

SET

На экран выводится список параметров в алфавитном порядке.

Нас интересует переменная:

OMP_NUM_THREADS.

Название можно расшифровать так:

OpenMP Number of Threads — Число потоков параллельной области OpenMP.

С списке переменных такого параметра пока нет (рис. 3.19).

Зато есть другой интересный параметр:

NUMBER_OF_PROCESSORS=8.

Как мы уже говорили, речь здесь идёт о так называемых «виртуальных процессорах», а вовсе не о микросхемах под названием «процессор».

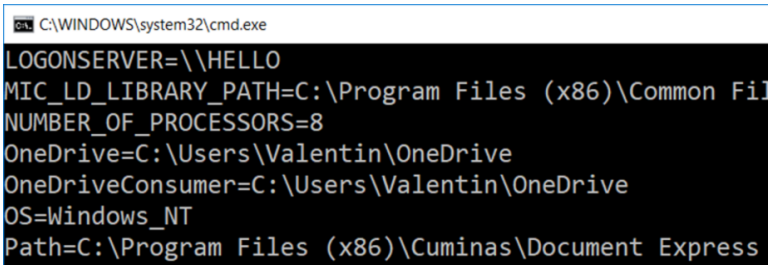


Рис. 3.19. Параметры окружения

Задание. Просмотрите список параметров окружения и обратите внимание на количество процессоров.

Если наш параметр не задан, по умолчанию создаётся количество параллельных потоков, равное числу виртуальных процессоров.

Зададим создание трёх потоков:

set OMP_NUM_THREADS=3

Проверим, что появилось в нашем списке.

Вводим команду

SET

У нас появился наш параметр (рис. 3.20).

```
C:\Users\Valentin>set OMP_NUM_THREADS=3

C:\Users\Valentin>set

MIC_LD_LIBRARY_PATH=C:\Program Files (x86)\Common
NUMBER_OF_PROCESSORS=8
OMP_NUM_THREADS=3
OneDrive=C:\Users\Valentin\OneDrive
OneDriveConsumer=C:\Users\Valentin\OneDrive
OS=Windows_NT
```

Рис. 3.20. Параметры окружения

Задание. Установите число потоков в командном окне.

Есть одна особенность работы с параметрами окружения в командной строке. Любые изменения параметров в командной строке действуют только в одном окне и только до тех пор, пока его не закроют. Постоянное и всеобщее задание параметров окружения делается по-другому. Мы этим заниматься не будем.

Задание. Откройте новое командное окно. Изучите список параметров окружения.

Нам нужно перейти в каталог, где находится скомпилированная программа — исполняемый файл.

Для этого придётся выяснить путь к файлу.

Возвращаемся в среду разработки.

Внизу экрана находим окно **Output** (рис. 3.21).

Здесь выводятся сообщения о результатах компиляции.

В нашем примере нам сообщают, что исполняемый файл успешно создан:

Build: 1 succeeded.

Нас интересует полный путь к файлу.:

E:\Arkov-OMP\omp-03-fork-join\Release\omp-03-fork-join. exe.

Путь можно скопировать в буфер, чтобы не печатать вручную.

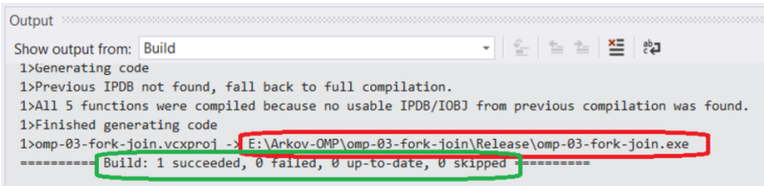


Рис. 3.21. Окно сообщений компилятора

Задание. Ознакомьтесь с окномсообщений о результатах компиляции программы.

Переходим в командное окно.

Мы находимся на диске **C:**, а исполняемый файл — на диске **E:**.

Для перехода на другой диск вводим его имя и двоеточие.

Обращаем внимание, как изменилось приглашение к вводу команд.

Теперь здесь имя другого диска (рис. 3.22).



Рис. 3.22. Смена текущего диска

Задание. Смените текущий диск.

Сменим текущий каталог.

Вводим команду Change Directory:

CD

и путь к файлу.

Проверяем содержимое каталога Directory:

DIR

Убеждаемся, что исполняемый файл находится в текущем каталоге.

Запускаем программу на выполнение.

Вводим имя файла и нажимаем **OK**.

Расширение **EXE** вводить не обязательно.

Видим, что параллельная область выполнялась на трёх потоках (рис. 3.23).

```
E:\>
E:\>cd E:\Arkov-OMP\omp-03-fork-join\Release\

E:\Arkov-OMP\omp-03-fork-join\Release>dir
 Volume in drive E is D0P0dataE
 Volume Serial Number is D053-A055

 Directory of E:\Arkov-OMP\omp-03-fork-join\Release

03/31/2020  09:49 PM    <DIR>          .
03/31/2020  09:49 PM    <DIR>          ..
03/31/2020  09:49 PM             9,216 omp-03-fork-join.exe
03/31/2020  09:49 PM             6,047 omp-03-fork-join.iobj
03/31/2020  09:49 PM             4,672 omp-03-fork-join.ipdb
03/31/2020  09:49 PM               319 omp-03-fork-join.log
03/31/2020  09:49 PM            73,504 omp-03-fork-join.obj
03/31/2020  09:49 PM           405,504 omp-03-fork-join.pdb
03/31/2020  09:49 PM    <DIR>          omp-03-fork-join.tlog
03/31/2020  09:49 PM            61,440 vc142.pdb
               7 File(s)        560,702 bytes
               3 Dir(s)   8,201,531,392 bytes free

E:\Arkov-OMP\omp-03-fork-join\Release>omp-03-fork-join
1
2
2
2
3

E:\Arkov-OMP\omp-03-fork-join\Release>
```

Рис. 3.23. Запуск программы

Задание. Запустите программу в командной строке и убедитесь, что вам удалось задать число потоков.

3.5.2. БИБЛИОТЕЧНАЯ ФУНКЦИЯ

Второй способ задать число параллельных потоков — это библиотечная функция. Конечно, речь здесь идёт о библиотеке OpenMP.

Формат вызова:

void omp_set_num_threads (int num).

Тип возвращаемого значения: **void**. Эта функция ничего не возвращает. Она просто делает своё дело.

Название функции можно расшифровать так:

Set Number of Threads — задать число параллельных потоков.

Чтобы использовать библиотечную функцию, нужно подключить эту библиотеку (рис. 3.24).

В начале программы указываем заголовочный файл:

#include <omp. h>.

E:\Arkov-OMP\omp-04-set\omp-04-set.c

```
 1  #include <stdio.h>
 2  #include <omp.h>
 3  void main()
 4  {
 5  omp_set_num_threads(2);
 6  printf("1\n");
 7  #pragma omp parallel
 8      printf("2\n");
 9  printf("3\n");
10  }
```

Рис. 3.24. Задание числа потоков

Задаём два параллельных потока.

Запускаем программу из среды разработки.

Видим, что в параллельной области сработало два потока (рис. 3.25).

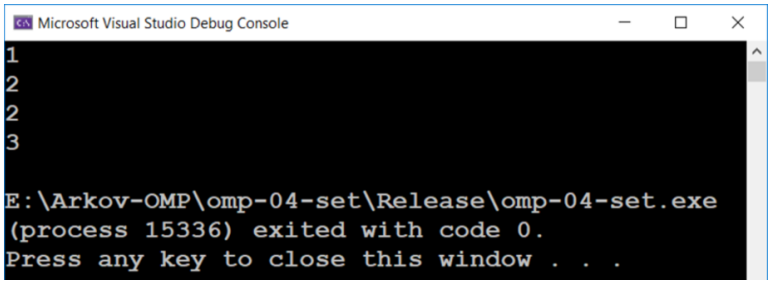


Рис. 3.25. Два потока

Задание. Установите число потоков с помощью и библиотечной функции и убедитесь, что эта настройка сработала.

3.5.3. ПАРАМЕТР ДИРЕКТИВЫ

Третий способ задать число параллельных потоков — указать в параметрах директивы **parallel**:

num_threads (num).

Составляем новую программу на основе предыдущей.

Указываем число потоков — четыре (рис. 3.26):

num_threads (4).

E:\Arkov-OMP\omp-05-num\omp-05-num.c

```
1  #include <stdio.h>
2  #include <omp.h>
3  void main()
4  {
5  printf("1\n");
6  #pragma omp parallel num_threads(4)
7      printf("2\n");
8  printf("3\n");
9  }
```

Рис. 3.26. Число потоков в директиве

Запускаем программу.

Убеждаемся, что число параллельных потоков соответствует тому, что мы задали (рис. 3.27).

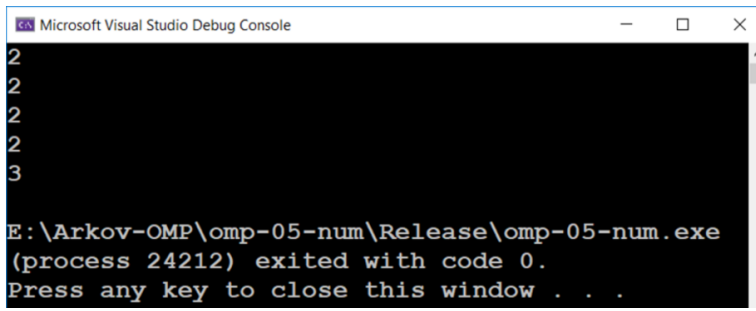


Рис. 3.27. Четыре потока

Задание. Установите число параллельных потоков через параметр директивы.

3.5.4. Приоритет настроек

Мы рассмотрели три способа задать число параллельных потоков.

Для одной программы можно использовать два или три способа одновременно.

При этом существует приоритет (относительная важность) настроек. Какая-то настройка важнее другой, и она отменяет менее важную настройку, заменяет своим параметром.

Задание. Проверьте все комбинации настроек и сделайте выводы о приоритете настроек.

4. УПРАВЛЕНИЕ РАСПАРАЛЛЕЛИВАНИЕМ

Мы познакомились с основными инструментами создания параллельных программ. Теперь немного углубимся в управление параллельными потоками.

4.1. ВЫЧИСЛИТЕЛЬНАЯ СРЕДА

Чтобы эффективно распараллелить решение задачи, нужно разделить большую задачу на части и раздать эти части параллельным потокам. При этом и главный поток, и каждый поток в группе должны в момент выполнения сориентироваться в обстановке. Нам нужно изнутри программы получить определённые сведения, например, такие:

— число процессоров в компьютере;
— число потоков в группе;
— номер текущего потока в группе.

4.1.1. ЧИСЛО ПРОЦЕССОРОВ

Начнём с числа процессоров.

Напомним, что слово «процессор» в нашем случае — это указание на «виртуальный процессор». Другими словами, это любой вычислительный элемент, который может выполнять один поток. Физическая реализация сейчас не важна.

Число «процессоров» можно узнать с помощью библиотечной функции

int omp_get_num_procs (void).

Функция возвращает количество доступных процессоров. Доступных именно нашей программе, а точнее ПРОЦЕССУ.

В предыдущей работе мы уже немного познакомились с привязкой процесса к конкретным ядрам.

Создадим новую программу и познакомимся с описанной функцией (рис. 4.1).

E:\Arkov-OMP\omp-06-numproc\omp-06-numproc.c

```
1  #include <stdio.h>
2  #include <omp.h>
3  int main()
4  {
5      int Np;
6      Np = omp_get_num_procs();
7      printf("Number of Processors = %d\n", Np);
8  }
9
```

Рис. 4.1. Число доступных процессоров

Запускаем программу и видим сообщение (рис. 4.2).

Убеждаемся, что заявленное число процессоров соответствует действительности.

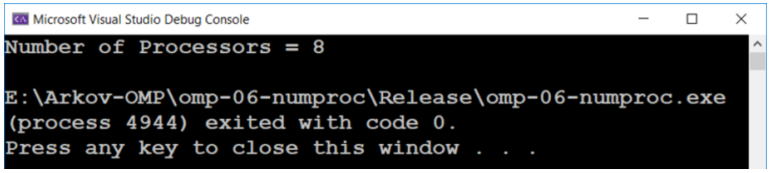


Рис. 4.2. Число процессоров

Задание. Создайте и запустите программу (рис. 4.1). Проверьте, как изменится сообщение программы, если установить число процессоров через параметр окружения **NUMBER_OF_PROCESSORS** и запустить программу в командной

строке.

4.1.2. ЧИСЛО ПОТОКОВ

Рассмотрим, как узнать число потоков в текущей параллельной области программы.

Для организации параллельной работы нужно знать, на сколько частей делить задачу. И вообще, надо следить за событиями. Даже если всё как бы само собой делается, за результаты отвечает почему-то не компьютер, а программист.

Итак, вот библиотечная функция, которая сообщает нам количество параллельных потоков в группе:

int omp_get_num_threads (void).

Расшифровывается название так:

Number of Threads — «число потоков».

Аргументов у функции нет, но скобки при вызове придётся использовать.

Вторая полезная библиотечная функция сообщает номер текущего потока в группе:

int omp_get_thread_num (void).

Название расшифровывается так:

Thread Number — «номер потока».

Номера потоков начинаются с нуля. Это будет главный поток.

Соответственно, максимальный номер потока равен числу потоков минус один.

Названия этих двух функций очень похожи, здесь просто другой порядок слов.

Судите сами:

num_threads / thread_num.

Придётся немного освежить английский. Здесь действует так называемое «правило ряда» (цепочка определений). Если идут несколько существительных подряд, то главное слово будет в конце.

Вторая трудность с пониманием названий — слово NUMBER. Оно может означать «количество» или «порядковый номер». Это одно и то же слово, но два разных значения. И переводятся разные значения разными словами. Вообще-то у этого слова ещё штук двадцать других значений, но дальше углубляться мы уже не будем.

Составим программу и посмотрим, как работают эти две функции (рис. 4.3).

Для организации вывода в виде таблицы используем символ табуляции:

\t.

Табуляция смещает начало вывода на 8 позиций.

Таким образом, первое число выводится начиная с 1-й позиции, второе число — с 9-й и т. д.

Табуляция нам ещё пригодится для анализа эффективности и точности.

E:\Arkov-OMP\omp-07-NumTread\omp-07-NumTh.c

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  void main()
5  {
6      printf("ThNum\tNumTh\n");
7  #pragma omp parallel
8      printf("%d\t%d\n", omp_get_thread_num(),
9              omp_get_num_threads());
10 }
```

Рис. 4.3. Номер потока и количество потоков

Запускаем программу и видим сообщения в произвольном порядке (рис. 4.4). Кто первый успел — тот и напечатал. Если запускать программу несколько раз, получим разные результаты.

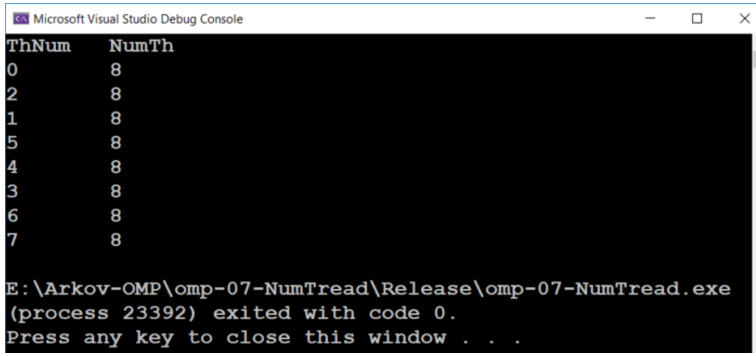


Рис. 4.4. Список номеров потоков

Задание. Составьте программу (рис. 4.3). Запустите программу несколько раз и обратите внимание на порядок вывода номеров потоков.

4.2. ПАРАЛЛЕЛЬНЫЙ ЦИКЛ

В этом разделе мы организуем параллельное исполнение цикла FOR и исследуем поведение программы. Результаты могут оказаться не совсем очевидными. Это особенности параллельного программирования. Вот с этими особенностями нам и предстоит познакомиться.

4.2.1. СИТУАЦИЯ ГОНКИ

При выполнении параллельной программы может возникать так называемая «ситуация гонки». Это соревнование параллельных потоков за доступ к данным. Английское название:

Data Race.

Мы её уже рассматривали в предыдущей работе. Параллельные потоки обращаются к общей переменной и «затира-

ют» чужие результаты, записывая свои. В результате появляется ошибка в расчётах. Причём ошибка будет случайной, непредсказуемой, и при каждом запуске ошибка будет разной.

Начнём с решения задачи распараллеливания цикла «в лоб».

Рассмотрим программу вычисления суммы большого количества единиц в цикле (рис. 4.5).

Перед оператором цикла вставляем следующую строчку:

#pragma omp parallel for (строка 6).

Эта директива организует параллельное выполнение разных итераций цикла несколькими потоками.

E:\Arkov-OMP\omp-08-race\omp-08-race.c

```
 1  #include <stdio.h>
 2  #include <omp.h>
 3  void main()
 4  {
 5      int i, S = 0;
 6  #pragma omp parallel for
 7      for (i = 0; i < 100000; i++)
 8          S++;
 9      printf("Sum = %d\n", S);
10  }
```

Рис. 4.5. Параллельный цикл

В нашей программе все параллельные потоки будут работать с одной общей переменной **S**. Для последовательной программы это не представляет каких-то проблем.

Запускаем командное окно и переходим в каталог с исполняемым файлом.

Задаём выполнение в один поток:

set OMP_NUM_THREADS=1.

Запускаем программу несколько раз (рис. 4.6).

Видим, что результат каждый раз получается правильный. Он соответствует количеству итераций цикла, заданному в нашей программе.

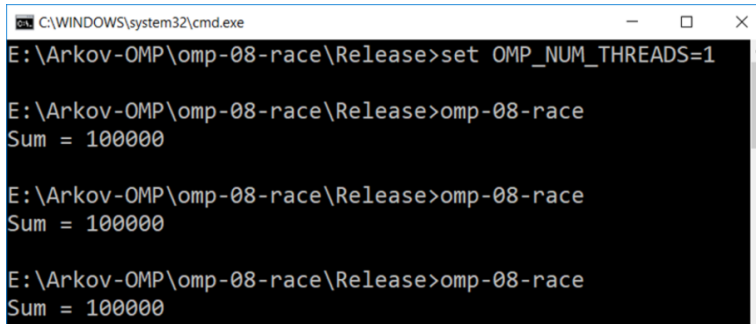


Рис. 4.6. Запуск одного потока

Задание. Создайте программу (рис. 4.5) и запустите её несколько раз. Обратите внимание на полученные результаты.

Попробуем запускать программу несколько раз, не загромождая экран лишним текстом. Для этого создадим пакетный файл (рис. 4.7).

Напомним, что пакетный файл — это текстовый файл, в котором вызываются команды операционной системы. То, что мы обычно набираем в командной строке вручную. Здесь также можно использовать дополнительные команды, напоминающие привычный язык программирования.

В первой строчке пакетного файла мы отключаем вывод на экран вводимой команды:

@echo off.

В следующих строчках мы несколько раз запускаем нашу программу на выполнение. Расширение имени файла **EXE** не указываем.

Сохраняем файл в каталоге **Release** — там же, где находится наш исполняемый файл. Указываем имя файла **08** и расширение имени **CMD**. Это сокращение от слова **COMMAND**. Можно также использовать традиционное расширение **BAT** — от слова **BATCH** — «пакетный файл».

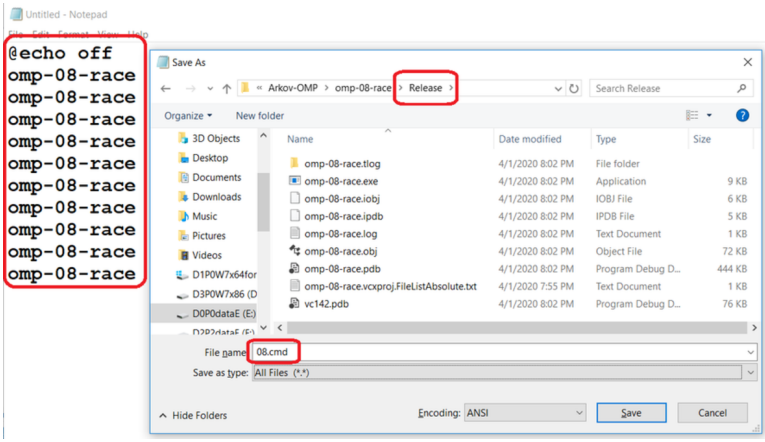


Рис. 4.7. Пакетный файл

Задание. Создайте пакетный файл (рис. 4.7).

Возвращаемся в командное окно. Переходим в каталог **Release** нашего текущего проекта (рис. 4.8). Вводим команду **dir**. Убеждаемся, что в текущем каталоге появился пакетный файл **08.cmd**. Здесь же видим нас исполняемый файл с расширением **EXE**.

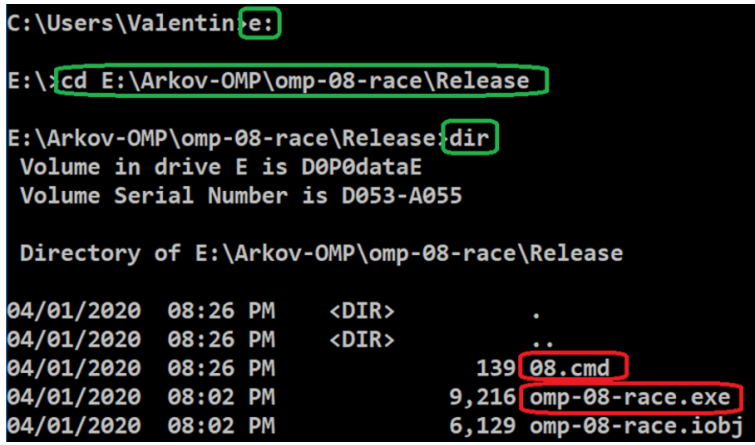


Рис. 4.8. Каталог**Release**

Запускаем наш пакетный файл. Вводим название файла, не указывая его расширение:

08.

Программа запускалась десять раз. Во всех случаях ответ правильный (рис. 4.9).

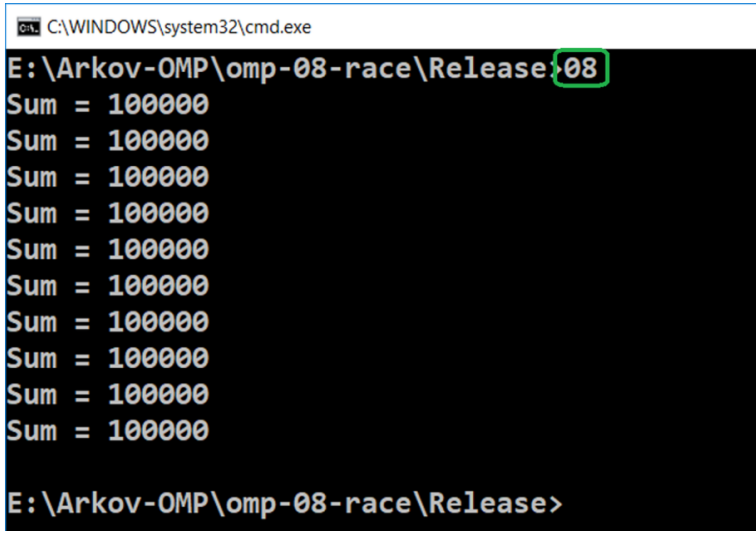


Рис. 4.9. Пакетный запуск программы

Задание. Запустите пакетный файл и изучите полученные результаты.

Мы создали первый пакетный файл и ввели нашу команду несколько раз. В пакетном файле можно организовать цикл. Традиционный цикл **FOR** со счётчиком. Тогда файл получается покороче и попроще.

Первая строчка отменяет вывод выполняемых команд на экран — как в предыдущем примере:

@echo off.

Вторая строчка — цикл **FOR**.

Имя счётчика цикла в пакетном файле начинается с двух символов процента:

%%i.

Ключ **/L** позволяет указать диапазон значений счётчика

и шаг изменения.

Далее мы указываем диапазон значений — от единицы до десяти с единичным шагом:

(1, 1, 10).

Тело цикла — наша команда — запуск программы на выполнение.

Запускаем пакетный файл (рис. 4.10).

Результаты нас устраивают.

Последовательное выполнение программы не даёт ошибок в расчётах.



Рис. 4.10. Цикл в пакетном файле

Задание. Создайте пакетный файл (рис. 4.10) и запустите его.

Теперь запустим несколько потоков.

Установим три потока:

set OMP_NUM_THREADS=3.

Запускаем пакетный файл.

Получаем десять разных результатов (рис. 4.11).

За возникновение таких нежелательных ситуаций отвечает программист. Так что составление параллельных программ требует дополнительного обучения.

```
E:\Arkov-OMP\omp-08-race\Release>set OMP_NUM_THREADS=3

E:\Arkov-OMP\omp-08-race\Release>008
Sum = 67237
Sum = 69755
Sum = 67100
Sum = 72090
Sum = 72766
Sum = 69369
Sum = 66890
Sum = 67050
Sum = 66808
Sum = 69890
```

Рис. 4.11. Результаты гонки

Задание. Запустите пакетный файл, задавая разное количество параллельных потоков. Обратите внимание на ошибки в расчётах.

4.2.2. РЕДУКЦИЯ

Более грамотный стиль параллельного программирования включает использование специальных возможностей. Мы рассмотрим одну из них.

При параллельном решении большой задачи возникает проблема «сбора промежуточных (частичных) сумм». Каждый поток обсчитывает свою часть задачи и получает часть общей суммы. После этого нужно собрать все частичные суммы и сложить их. Такая операция нужна очень часто. Поэтому была разработана готовая процедура «редукции».

Английское слово REDUCTION в данном конкретном случае означает «сворачивание, собирание, соединение, объединение, сведение».

Мы будем использовать параметр reduction в директиве **pragma**:

#pragma omp parallel for reduction (+: S).

Эта конструкция позволяет автоматически организовать параллельные вычисление и сбор частичных сумм.

Знак **[+]** указывает, что будет найдена сумма (есть и другие операции).

S — переменная, с которой будем работать.

Составляем программу (рис. 4.12).

При входе в параллельную область в каждом потоке создаётся локальная копия переменной **S**. Это механизм напоминает создание локальной переменной внутри подпрограммы. Из других модулей к локальной переменной доступа нет.

Слово ЛОКАЛЬНЫЙ буквально означает «местный, внутренний».

E:\Arkov-OMP\omp-09-reduce\omp-09-reduce.c

```
 1  #include <stdio.h>
 2  #include <omp.h>
 3  void main()
 4  {
 5      int i, S = 0;
 6  #pragma omp parallel for reduction(+:S)
 7      for (i = 0; i < 100000; i++)
 8          S++;
 9      printf("Sum = %d\n", S);
10  }
```

Рис. 4.12. Редукция

Создаём пакетный файл, аналогичный рассмотренному.

Переходим в командное окно и задаём разное количество параллельных потоков.

Запускаем пакетный файл.

Смотрим на результаты (рис. 4.13).

Последовательный запуск даёт правильные результаты.

```
E:\Arkov-OMP\omp-09-reduce\Release>set OMP_NUM_THREADS=1

E:\Arkov-OMP\omp-09-reduce\Release>009
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
```

Рис. 4.13. Последовательный запуск

Задаём несколько потоков.

Запуск тоже даёт правильные результаты (рис. 4.14).

Мы применили редукцию и попутно исключили ситуацию гонки.

```
E:\Arkov-OMP\omp-09-reduce\Release>set OMP_NUM_THREADS=4

E:\Arkov-OMP\omp-09-reduce\Release>009
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
Sum = 100000
```

Рис. 4.14. Параллельный запуск

Задание. Запустите параллельную программу с разным количеством потоков и изучите полученные результаты.

4.2.3. ПОРЯДОК ВЫПОЛНЕНИЯ ПОТОКОВ

Следующий эксперимент — изучим порядок запуска и выполнения параллельных потоков.

Составим программу на основе предыдущих (рис. 4.15). Обратите внимание, что мы временно убрали редукцию из директивы **parallel**. Все потоки обращаются к одной и той же общей переменной **S**.

Программа печатает заголовок таблицы (строка 6).

Затем выводит номер потока и значение суммы (строка 10).

Так мы сможем проследить, в каком порядке работает программа.

E:\Arkov-OMP\omp-10-order\omp-10-order.c

```
 1  #include <stdio.h>
 2  #include <omp.h>
 3  void main()
 4  {
 5      int i, S = 0;
 6      printf("ThNum\tS\n");
 7  #pragma omp parallel for
 8      for (i = 0; i < 10; i++) {
 9          S++;
10          printf("%d\t%d\n", omp_get_thread_num(), S);
11      }
12  }
```

Рис. 4.15. Порядок потоков

Запускаем программу, получаем таблицу (рис. 4.16).

Как видим, номера потоков печаются в случайном порядке.

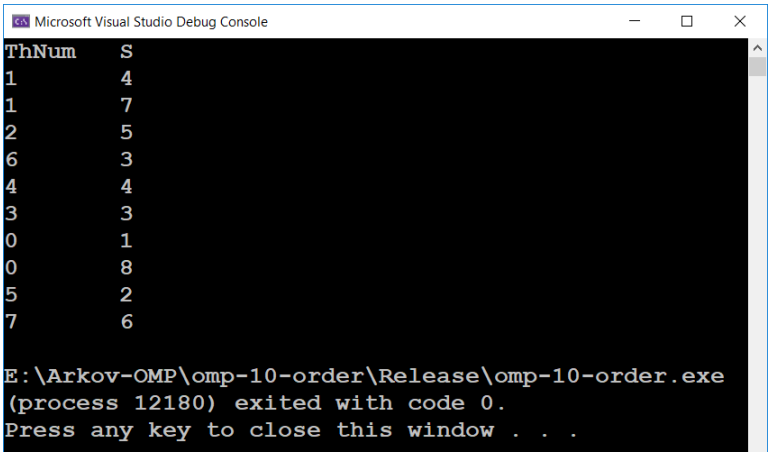


Рис. 4.16. Порядок потоков

Задание. Создайте программу (рис. 4.15) и запустите её. Обратите внимание на порядок выполнения потоков.

Перейдём в командное окно.

Запускаем нашу программу и направим вывод в файл — вместо вывода на экран:

omp-10-order> order. txt.

Эта манипуляция называется «перенаправление стандартного вывода». Экран — это стандартное устройство вывода, с которым работает функция **printf**. Вместо экрана, весь текстовый вывод отправляется в файл.

Итак, теперь вся таблица вместе с заголовком записана в текстовый файл. Открываем файл в **Блокноте** и убеждаемся в том, что вся информация здесь (рис. 4.17).

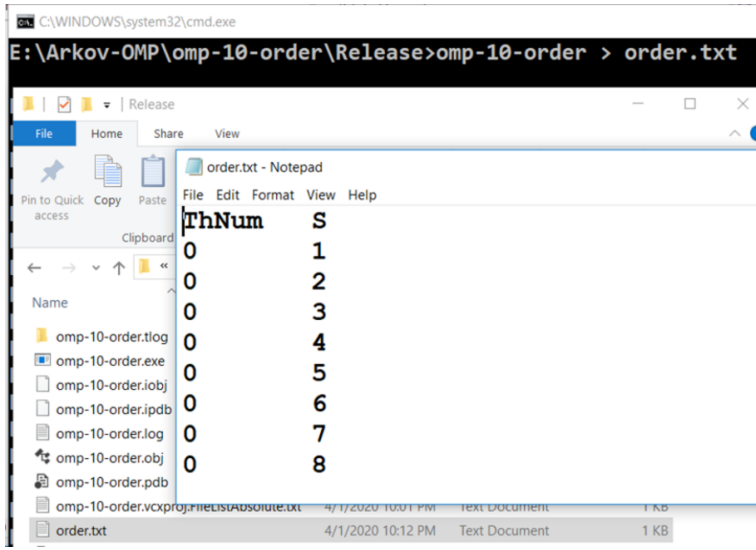


Рис. 4.17. Вывод в файл

Задание. Запустите программу и направьте вывод в файл. Откройте файл в Блокноте и изучите его содержимое.

Загрузим текстовый файл для анализа в электронную таблицу.

Запускаем **Excel**. Вызываем в верхнем меню открытие файла: **File — Open — Other locations — Browse**.

Переходим в каталог **Release и находим текстовый файл: order. txt**.

Устанавливаем загрузку текстовых файлов:

Text Files (*prn, *.txt, *csv).

В правой части окна видим содержимое файла (рис. 4.18).

Нажимаем **OK**.

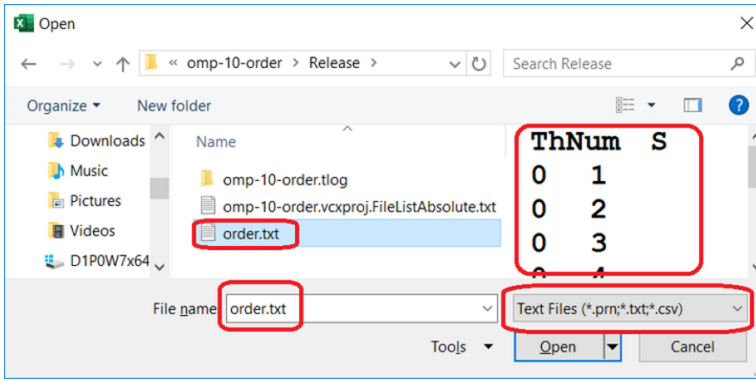


Рис. 4.18. Загрузка файла

В окне Мастера импорта файла настраиваем параметры:
Original data type — Delimited;
My data has headers (рис. 4.19).
Нажимаем **Next**.

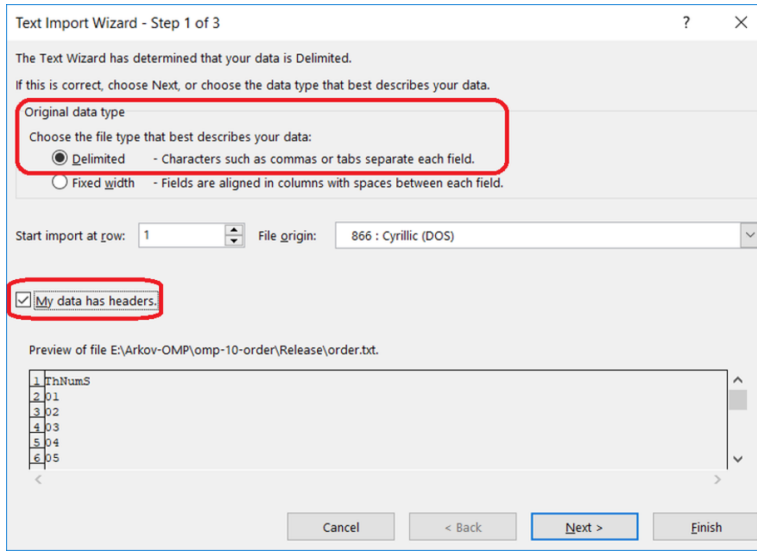


Рис. 4.19. Первый шаг импорта

На втором шаге импорта указываем разделитель полей — символ табуляции:

Delimiters — Tab.

Вспоминаем, что при выводе на экран мы использовали табуляцию.

Нажимаем **Next** (рис. 4.20).

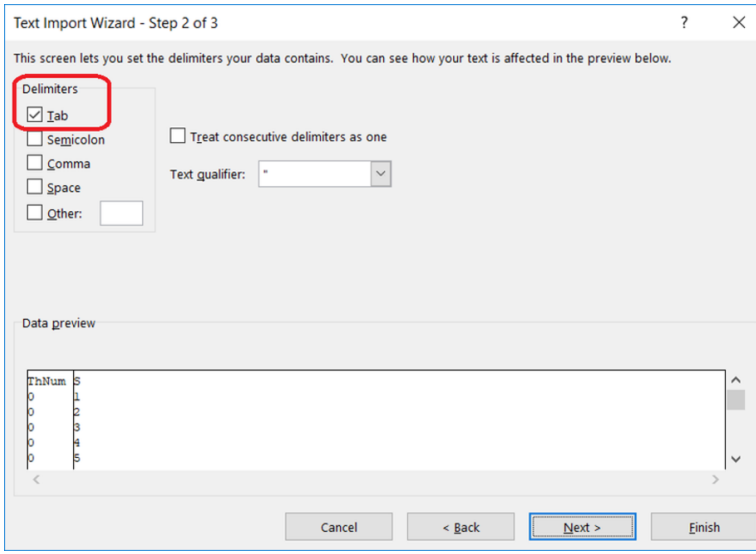


Рис. 4.20. Второй шаг импорта

После импорта файла получаем таблицу в **Excel** (рис. 4.21).
Выделяем диапазон ячеек — заголовок и данные.
Выбираем в верхнем меню форматирование таблицы:
Home — Styles — Format as Table.
Выбираем оформление таблицы.

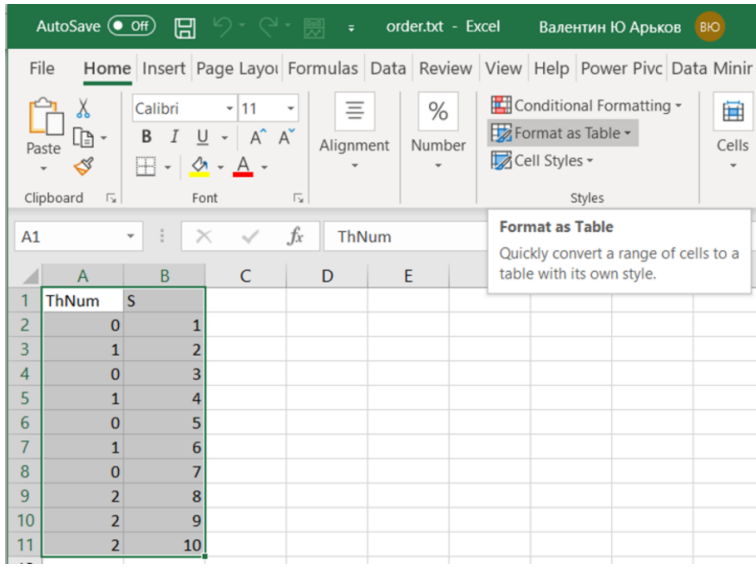


Рис. 4.21. Таблица из файла

Задание. Загрузите текстовый файл в **Excel**.

Сортируем таблицу по колонке **S** (рис. 4.22).

Для этого нажимаем кнопку в виде стрелочки справа от названия столбца. Выбираем в выпадающем списке сортировку по возрастанию:

Sort Smallest to Largest.

Изучаем порядок выполнения потоков.

Наблюдаем совершенно непредсказуемую картину.

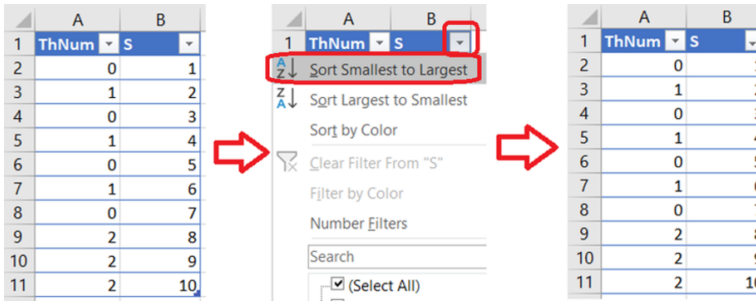


Рис. 4.22. Сортировка по **S**

Задание. Выполните сортировку и изучите порядок выполнения потоков.

4.2.4. ЛОКАЛЬНЫЕ КОПИИ ПЕРЕМЕННОЙ

Вернёмся к редукции и рассмотрим локальные копии общей переменной.

Итак, в директиве **parallel** мы организуем редукцию переменной **S** путём сложения частичных, промежуточных значений: **reduction (+:S)**.

Это означает, что при входе в параллельную область создаётся несколько параллельных потоков. В каждом из этих потоков (включая и главный поток) берётся наша общая переменная **S** и для неё создаётся локальная копия. То есть в каждом потоке появляется своя внутренняя переменная.

Мало того, в момент создания локальная копия ещё и получает начальное значение. Это называется ИНИЦИАЛИЗАЦИЯ — «присвоение начального значения». Если редукция делается путём суммирования, то локальные копии получают нулевое начальное значение.

Немного изменим предыдущею программу (рис. 4.23).

Перед началом параллельной области присвоим общей переменной **S** начальное значение **10** (строка 5).

Добавляем редукцию для параллельной области (строка 7).

В цикле делаем 20 итераций, чтобы более наглядно показать процесс выполнения программы (строка 8).

После завершения параллельной области и редукции выводим собранное из локальных копий суммарное значение общей переменной **S** (строка 12).

На экран дополнительно выводим столбец значений счётчика цикла **i** (строки 6, 10, 12).

После завершения цикла в колонке значений счётчика выводим число 20 (строка 12). Обратим внимание, что при выполнении цикла счётчик принимает значения от **0** до **19**.

E:\Arkov-OMP\omp-11-local\omp-11-local.c

```
 1  #include <stdio.h>
 2  #include <omp.h>
 3  void main()
 4  {
 5      int i, S = 10;
 6      printf("i\tThNum\tS\n");
 7  #pragma omp parallel for reduction(+:S)
 8      for (i = 0; i < 20; i++) {
 9          S++;
10          printf("%d\t%d\t%d\n", i, omp_get_thread_num(), S);
11      }
12      printf("%d\t%d\t%d\n", 20, omp_get_thread_num(), S);
13  }
```

Рис. 4.23. Отслеживание инициализации

Далее проделаем знакомые нам шаги по выполнению программы и анализу результатов в **Excel**.

Компилируем программу и убеждаемся в отсутствии ошибок.

Переходим в командное окно.

Задаём три потока:

set OMP_NUM_THREADS=3.

Запускаем программу на выполнение.

Перенаправляем вывод в текстовый файл.

Загружаем файл в **Excel**.

Форматируем таблицу и сортируем по номеру потока (рис. 4.24).

Теперь можно немного отдышаться и рассмотреть результаты работы.

При входе в параллельную область каждая локальная копия переменной **S** получила нулевое значение. На каждой итерации цикла к ней прибавляли по единичке.

Нулевой и первый потоки получили в своё распоряжение по семь итераций, второй поток — шесть. Так что работу поделили почти поровну.

После выхода из параллельной области частичные суммы сложили, да ещё к этой сумме добавили первоначальное значение 10:

S = 7 +7 +6 +10 = 30.

Так работает редукция в параллельном цикле **for**.

	A	B	C
1	i	ThNum	S
2	0	0	1
3	1	0	2
4	2	0	3
5	3	0	4
6	4	0	5
7	5	0	6
8	6	0	7
9	20	0	30
10	7	1	1
11	8	1	2
12	9	1	3
13	10	1	4
14	11	1	5
15	12	1	6
16	13	1	7
17	14	2	1
18	15	2	2
19	16	2	3
20	17	2	4
21	18	2	5
22	19	2	6

Рис. 4.24. Изменение локальных копий

Задание. Составьте программу (рис. 4.23) и изучите поведение локальных копий общей переменной при распараллеливании цикла с редукцией.

4.3. ВЫЧИСЛИТЕЛЬНАЯ НАГРУЗКА

В этом разделе мы познакомимся с тем, как распределяется вычислительная нагрузка между виртуальными процессорами. Проще говоря, между ядрами процессора.

Наша конечная цель — загрузить все ядра работой на 100%. Только тогда можно будет измерить эффективность распараллеливания.

4.3.1. ПОЛНАЯ ЗАГРУЗКА

Первый эксперимент — полная загрузка процессора.

Чтобы обеспечить работой все вычислительные элементы процессора, сделаем побольше итераций в нашем цикле. А для этого придётся взять другой целый тип счётчика — «длинное-длинное целое»:

long long.

Это восемь байт, или 8 * 8 = 64 бита. Максимальное значение соответствует $\mathbf{2^{64}}$. Для целого числа со знаком получится раза в два поменьше, но всё равно хорошо.

Текст программы с нашей «большой задачей» приведён на рис. 4.25.

Мы объявляем тип счётчика и суммы (строка 4).

Число итераций нашего цикла берём много, но не больше максимально возможного (строка 7).

Тело цикла — прибавление единичек к сумме (строка 8).

E:\Arkov-OMP\omp-12-load\imp-12-load.c

```
 1  #include <stdio.h>
 2  #include <omp.h>
 3  void main() {
 4      long long i, S = 0;
 5      printf("S = %lld\n", S);
 6  #pragma omp parallel for reduction(+:S)
 7      for (i = 0; i < 1000000000000; i++)
 8          S++;
 9      printf("S = %lld\n", S);
10  }
```

Рис. 4.25. «Большая» задача

Компилируем программу и переходим в командную строку.

Задаём количество потоков, равное количеству ядер.

Открываем **Диспетчер задач**:

Task Manager.

Находим вкладку **Быстродействие**:

Performance.

Запускаем программу на выполнение и смотрим на загрузку ядер (рис. 4.26).

Каждое ядро загружено на 100%.

Кроме того, в левой части окна показан график общей загрузки процессора в целом — тоже 100%.

Переходим в командное окно и нажимаем комбинацию клавиш для прерывания программы:

Ctrl + C.

Загрузка процессора снова падает почти до нуля.

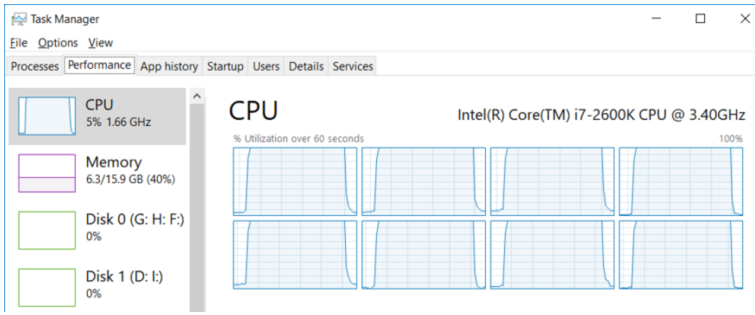


Рис. 4.26. Полная загрузка всех ядер

Задание. Составьте программу (рис. 4.25) и проведите эксперимент с полной загрузкой всех ядер процессора (рис. 4.26).

4.3.2. НЕПОЛНАЯ ЗАГРУЗКА

Проведём следующий эксперимент.

Уменьшим количество потоков до двух.

Запускаем программу и следим за загрузкой процессора (рис. 4.27).

Видим, что на этот раз все ядра «немного заняты» работой.

Мы запустили задачу на двух параллельных потоках. Однако, операционная система старается равномерно распределить вычислительную нагрузку между всеми ядрами.

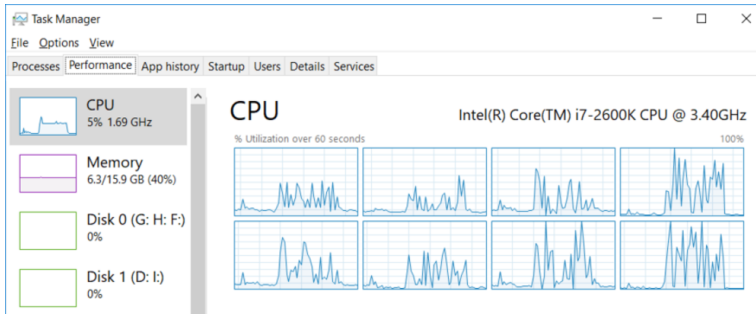


Рис. 4.27. Частичная загрузка всех ядер

Задание. Проведите эксперимент с частичной загрузкой всех ядер процессора, задав число потоков меньше, чем число ядер.

4.3.3. ПРИВЯЗКА К ЯДРАМ

Привяжем параллельные потоки к конкретным ядрам процессора. Добавим в нашу программу с большой задачей несколько строк (рис. 4.28).

Вставляем дополнительные команды в начале параллельной области, когда локальная копия **S** получает нулевое значение (строка 11).

Определяем номер текущего потока:

omp_get_num_thread ().

Затем формируем маску для привязки потока к ядру (строка 13):

Mask = pow (2, TNum).

Возводим два в степень, равную номеру потока. Номера потоков начинаются от нуля. Маска привязки к вычислительному ядру состоит из степеней двойки. Таким образом, мы привязываем каждый поток к своему ядру.

Например, нулевой поток привязываем к нулевому ядру. Маска равна

$2^0 = 1$.

Первый поток привязываем к первому ядру. Маска равна **$2^1 = 2$**.

И так далее.

Затем в строке 14 задаём привязку к ядру с помощью следующей функции:

SetThreadAffinityMask.

Для контроля выводим данные на экран (строка 15).

В нашей программе появился новый параметр директивы parallel:

private (TNum, Mask).

Таким способом мы объявляем, что две переменные будут локальными внутри параллельных потоков. Это означает, что в каждом потоке будет своя переменная с указанным названием. Присвоение значения такой переменной в одном потоке никак не повлияет на значение той же самой переменной в другом потоке. В нашей программе это номер потока в группе и маска для привязки к ядру.

E:\Arkov-OMP\omp-13-mask\omp-13-mask.c

```
 1  #include <stdio.h>
 2  #include <omp.h>
 3  #include <windows.h>
 4  void main()
 5  {
 6      int TNum, Mask;
 7      long long i, S = 0;
 8      printf("S = %lld\n", S);
 9  #pragma omp parallel for reduction(+:S) private (TNum, Mask)
10      for (i = 0; i < 1000000000000; i++) {
11          if (S == 0) {
12              TNum = omp_get_thread_num();
13              Mask = pow (2, TNum);
14              SetThreadAffinityMask(GetCurrentThread(), Mask);
15              printf("TNum = %d\tMask = %d\n", TNum, Mask);    }
16          S++;
17      }
18      printf("S = %lld\n", S);
19  }
```

Рис. 4.28. Привязка потоков к ядрам

Компилируем.

Переходим в командное окно. Задаём два потока и запускаем программу.

Рассмотрим загрузку в Диспетчере задач (рис. 4.29).

Теперь нулевое и первое ядра загружены на 100%. Остальные ядра не участвуют в нашей работе.

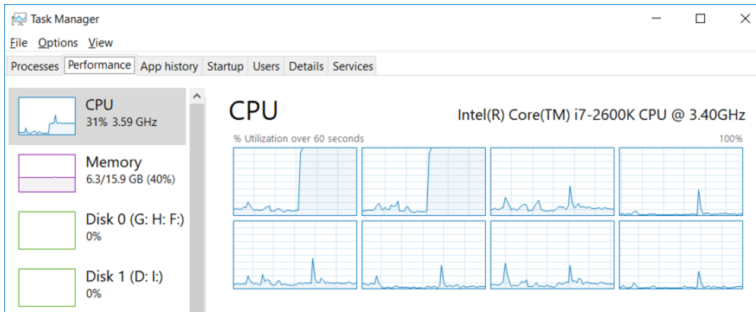


Рис. 4.29. Два потока — два ядра

Задание. Составьте программу (рис. 4.28) и запускайте её с разным количеством потоков. Обратите внимание на загрузку отдельных ядер.

Рассмотрим значения маски для разного количества потоков. Убедимся, что значения маски вычисляются правильно (рис. 4.30).

```
E:\Arkov-OMP\omp-13-mask\Release>set OMP_NUM_THREADS=4

E:\Arkov-OMP\omp-13-mask\Release>omp-13-mask
S = 0
TNum = 2        Mask = 4
TNum = 0        Mask = 1
TNum = 1        Mask = 2
TNum = 3        Mask = 8
^C
E:\Arkov-OMP\omp-13-mask\Release>set OMP_NUM_THREADS=2

E:\Arkov-OMP\omp-13-mask\Release>omp-13-mask
S = 0
TNum = 0        Mask = 1
TNum = 1        Mask = 2
S = 1000000000000
```

Рис. 4.30. Вычисление маски

Задание. Установите разные значения количества потоков и проверьте правильность вычисления маски.

4.4. ВРЕМЯ ВЫЧИСЛЕНИЙ

Мы рассмотрели организацию параллельных вычислений. Настало время заняться временем. Нас интересует время вычислений. По нему мы сможем оценить эффективность распараллеливания.

Время в компьютере измеряют с помощью системного таймера. Для обращения к таймеру есть ряд готовых функций.

В предыдущей работе мы использовали стандартную функцию **clock ()**.

Чтобы получить время в секундах делим разность показаний на количество тиков **CLOCKS_PER_SEC**.

В библиотеке OpenMP есть своя функция для работы с таймером:

double omp_get_wtime (void).

Точность измерения времени можно узнать с помощью функции:

double omp_get_wtick (void).

Обе функции дают результат в секундах.

Возьмём предыдущую программу с «большой задачей» суммирования. Добавим замеры времени и выведем результаты на экран (рис. 4.31).

E:\Arkov-OMP\omp-14-time\omp-14-time.c

```c
#include <stdio.h>
#include <omp.h>
#include <time.h>
void main()
{
    long long i, S = 0;
    double T;
    printf("S = %lld\n", S);
    clock_t t;
    double tt;
    t = clock();
    tt = omp_get_wtime();
#pragma omp parallel for reduction(+:S)
    for (i = 0; i < 100000000000; i++)
        S++;
    t = clock() - t;
    tt = omp_get_wtime() - tt;
    T = (float) t / CLOCKS_PER_SEC;
    printf("S = %lld\n", S);
    printf("T_win = %2.10f c\tTick = %2.10f c\n",
        T, 1./ (float) CLOCKS_PER_SEC);
    printf("T_omp = %2.10f c\tTick = %2.10f c\n",
        tt, omp_get_wtick());
}
```

Рис. 4.31. Замеры времени

Запускаем программу и подбираем число итераций цикла так, чтобы работа продолжалась примерно секунд 20.

Нас интересует, насколько существенной будет «разница в показаниях». В нашем примере разница в третьем-четвёртом знаке после запятой, или в пятом значащем разряде (рис. 4.32).

```
E:\Arkov-OMP\omp-14-time\Release>set OMP_NUM_THREADS=8

E:\Arkov-OMP\omp-14-time\Release>omp-14-time
S = 0
S = 100000000000
T_win = 17.3010005951 c Tick = 0.0010000000 c
T_omp = 17.3006355991 c Tick = 0.0000003012 c
```

Рис. 4.32. Две оценки времени

Зададим побольше итераций. Разница по-прежнему в третьем-четвёртом знаке после запятой (рис. 4.33). И это приемлемая разница — если принять во внимание, что заявленная точность библиотечной функции **clock ()** составляет 0,001 сек.

В следующих упражнениях мы примем дополнительные меры для повышения точности измерения времени.

```
E:\Arkov-OMP\omp-14-time\Release>omp-14-time
S = 0
S = 1000000000000
T_win = 161.6990051270 c        Tick = 0.0010000000 c
T_omp = 161.6988373452 c        Tick = 0.0000003012 c
```

Рис. 4.33. Ещё две оценки

Задание. Составьте и запустите программу (рис. 4.31). Сравните значения продолжительности вычислений, полученные двумя способами.

5. ПАРАМЕТРЫ КОМАНДНОЙ СТРОКИ

Для организации экспериментов нам понадобится много раз запускать одну и ту же программу с разными настройками и параметрами. Конечно, можно каждый раз изменять исходный текст, компилировать исполняемый файл и вручную запускать его на выполнение. Но это дополнительные затраты времени. Мы поступим по-другому.

В нашем распоряжении есть такой инструмент, как параметры (аргументы) командной строки. Мы его уже использовали в предыдущей работе. Напомним, что это такое и как это работает.

В командной строке мы запускаем программу на выполнение. Мы пишем название файла. Расширение имени файла можно не указывать. После имени файла через пробел можно указать дополнительные параметры. Это просто символы, строки текста, разделённые пробелами. Программа может прочитать их — точно так же, как мы вводим данные с клавиатуры. Этот приём часто используется при вызове команд операционной системы.

Задание. Изучите в Википедии статью «Интерфейс командной строки». Обратите внимание на раздел «Формат команды» и выясните, что означают квадратные скобки в описании команд операционной системы. Обратите внимание на раздел «Достоинства» и выясните, какое преимущество командной строки мы используем для организации наших вычислительных экспериментов.

5.1. КОМАНДА DIR

Рассмотрим для примера работу с командой **DIR**. Эта команда выводит на экран содержимое директории, то есть список файлов в текущем каталоге. Название команды — это сокращение английского слова DIRECTORY.

Задание. Прочитайте в английской версии Википедии начало статьи «Directory (computing)» и выясните, что такое «директория». Переключитесь на русский вариант статьи и выясните, какими словами может называться этот объект.

Познакомимся с командой DIR. Переходим в командное окно и вводим нашу команду. Сейчас мы вводим саму команду без параметров (рис. 5.1). Получаем сведения про текущий диск, папку и файлы.

```
E:\Arkov-OMP\omp-15-argv\Release>dir
 Volume in drive E is D0P0dataE
 Volume Serial Number is D053-A055

 Directory of E:\Arkov-OMP\omp-15-argv\Release

04/03/2020  11:38 AM    <DIR>          .
04/03/2020  11:38 AM    <DIR>          ..
04/03/2020  11:38 AM             9,216 omp-15-argv.exe
04/03/2020  11:38 AM             4,981 omp-15-argv.iobj
04/03/2020  11:38 AM             3,936 omp-15-argv.ipdb
04/03/2020  11:38 AM               299 omp-15-argv.log
04/03/2020  11:38 AM            73,114 omp-15-argv.obj
04/03/2020  11:38 AM           397,312 omp-15-argv.pdb
04/03/2020  11:38 AM    <DIR>          omp-15-argv.tlog
04/03/2020  11:38 AM                50 omp-15-argv.vcxproj.FileListAbsolute.txt
04/03/2020  11:38 AM            61,440 vc142.pdb
               8 File(s)        550,348 bytes
               3 Dir(s)   3,766,198,272 bytes free
```

Рис. 5.1. Команда DIR без параметров

Чтобы узнать подробности о нашей команде, вызываем её с параметром **/?** (рис. 5.2). То, что вводят после имени команды, называют разными словами: параметры, ключи, опции и т. п.

Нам сообщают, что делает наша команда, какой у неё формат вызова и что делают разные ключи. Видим, что параметры нужно вводить через пробел после команды.

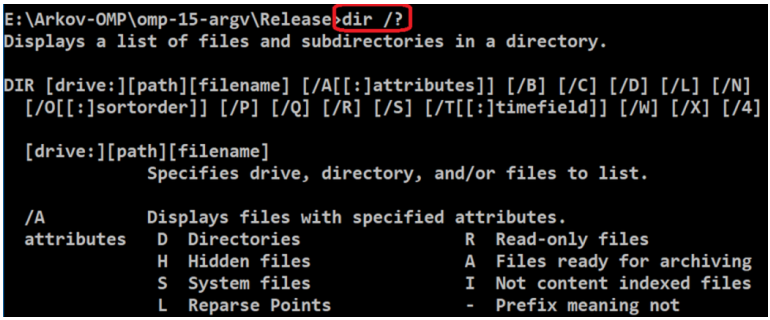


Рис. 5.2. Справка по использованию команды

Задание. Вызовите справку и выясните, что делают параметры **/B**, **/O** и **/X**. Вызовите команду **DIR** по очереди с каждым из этих ключей.

5.2. ЧТЕНИЕ ПАРАМЕТРОВ

При запуске программы на выполнение можно получить параметры командной строки (command-line arguments). Для этого в начале программы в заголовке главной функции main указывают аргументы **argc** и **argv**:

int main (int argc, char* argv []).

argc — сокращение от выражения arguments count (the count of arguments). Это количество параметров командной строки. Если argc равен 1, значит при вызове вводили только имя исполняемого файла.

argv — сокращение от arguments values (the values of the arguments). Это массив указателей на строки символов. Каждый аргумент командной строки — это строка, состоящая из одного или нескольких символов. Квадратные скобки после argv гово-

рят о том, что это массив. Звёздочка — что это указатель. Таким образом, argv указывает на первый элемент массива. А размер этого массива мы узнаём из предыдущего параметра **argc**.

Напомним, что нумерация элементов массива в языке программирования Си начинается с нуля. Таким образом, **argv [0]** — это имя запущенной программы (файла типа **EXE**), **argv [1]** — это первый аргумент (параметр) после имени файла и т. д.

Составим программу, которая читает параметры командной строки и выводит их на экран (рис. 5.3).

Вначале мы выводим на экран количество аргументов (строка 5). Это значение переменной **argc**.

Затем мы организуем цикл от **0** до **(argc — 1)** и выводим номер и значение аргументов. Для вывода строки символов на экран мы используем формат **%s** — от слова string — «строка».

E:\Arkov-OMP\omp-15-argv\omp-15-argv.c

```
1  #include <stdio.h>
2  void main(int argc, char* argv[])
3  {
4      int i;
5      printf("argc = %d\n", argc);
6      for (i = 0; i < argc; i++) {
7          printf("argv[%d] = %s\n", i, argv[i]);
8      }
9  }
```

Рис. 5.3. Чтение и вывод параметров

Запускаем программу из среды разработки

Ctrl + F5.

Рассматриваем результаты (рис. 5.4).

У нас один параметр командной строки.

Получается, что исполняемый файл был запущен на выполнение с указанием полного пути, включая имя диска и все каталоги.

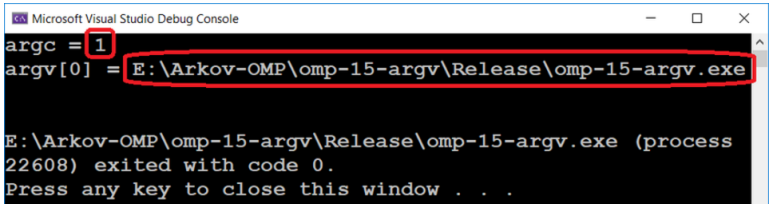


Рис. 5.4. Запуск из среды разработки

Задание. Составьте программу (рис. 5.3) и запустите её из среды разработки.

Среда разработки позволяет нам проводить отладку программы (debugging) с передачей параметров командной строки.

Открываем свойства проекта:

Project — Properties.

В окне параметров проекта переходим к следующему пункту:

Configuration Properties — Debugging — Command Arguments — Edit (рис. 5.5).

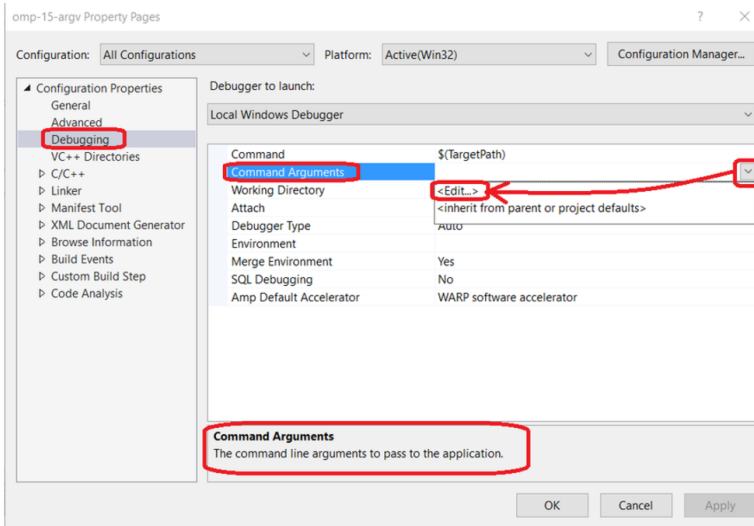


Рис. 5.5. Аргументы командной строки

Далее в окне **Command Arguments** вводим параметры, разделяя их пробелом (рис. 5.6).

Нажимаем **OK**.

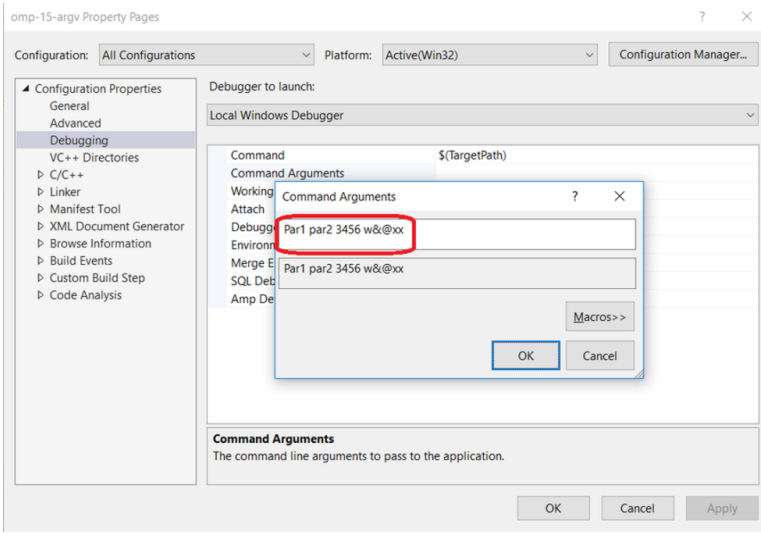


Рис. 5.6. Ввод аргументов

Снова запускаем программу из среды разработки.

Теперь на экран выводятся все наши параметры (рис. 5.7).

Убеждаемся, что программа распознала параметры и вывела их на экран.

Удаляем наши параметры в свойствах проекта.

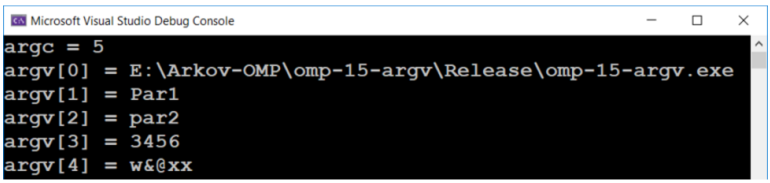


Рис. 5.7. Вывод параметров

Задание. Задайте несколько параметров командной строки

для отладки и запустите программу из среды разработки.

Мы убедились, что программа работает.

Запустим её в командном окне.

Указываем несколько параметров через пробел и рассматриваем результаты (рис. 5.8).

Среди прочего, мы видим, было ли указано расширение имени фала при запуске программы.

```
E:\Arkov-OMP\omp-15-argv\Release>omp-15-argv.exe
argc = 1
argv[0] = omp-15-argv.exe

E:\Arkov-OMP\omp-15-argv\Release>omp-15-argv
argc = 1
argv[0] = omp-15-argv

E:\Arkov-OMP\omp-15-argv\Release>omp-15-argv par1 123 wQ#@%xxx
argc = 4
argv[0] = omp-15-argv
argv[1] = par1
argv[2] = 123
argv[3] = wQ#@%xxx
```

Рис. 5.8. Вывод аргументов

Задание. Запустите программу в командном окне с несколькими аргументами и рассмотрите результаты.

5.3. ИСПОЛЬЗОВАНИЕ ПАРАМЕТРОВ

Мы научились читать параметры (аргументы) командной строки. Теперь разберёмся с тем, как делать «разбор» (parsing) этих строк. Нам нужно взять строку и «вытащить» из неё число.

Будем использовать функцию преобразования строки символов в целое число:

int atoi (const char *str).

Название функции **atoi** можно расшифровать так:

Convert «a» (string of symbols) to integer.

Текст программы приводится на рис. 5.9.

Мы возьмём первый параметр и преобразуем его в целое число (строка 4).

Затем выведем исходную строку символов (строка 5) и полученное число (строка 6).

В нашей программе мы не проверяем количество аргументов. Конечно же, в более серьёзных приложениях такая проверка нужна обязательно.

E:\Arkov-OMP\omp-16-atoi\omp-16-atoi.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  void main(int argc, char* argv[]) {
4      int N;
5      N = atoi( argv[1] );
6      printf("a = %s\n", argv[1]);
7      printf("N = %d\n", N);
8  }
```

Рис. 5.9. Преобразование строки в целое число

Запускаем программу и проверяем разные входные значения (рис. 5.10).

Напомним, что целое число типа **int** хранится в виде 4 байтов. Соответственно можно оценить диапазон значений для целого числа со знаком. Максимальное значение — это 2 в 31-й степени, то есть чуть больше двух миллиардов.

При выходе за пределы возможных значений мы получаем минимальное и максимальное число соответственно:

-2^{31} ... $2^{31}-1$.

```
E:\Arkov-OMP\omp-16-atoi\Release>omp-16-atoi 10
a = 10
N = 10

E:\Arkov-OMP\omp-16-atoi\Release>omp-16-atoi -10
a = -10
N = -10

E:\Arkov-OMP\omp-16-atoi\Release>omp-16-atoi 10000000000
a = 10000000000
N = 2147483647

E:\Arkov-OMP\omp-16-atoi\Release>omp-16-atoi -10000000000
a = -10000000000
N = -2147483648
```

Рис. 5.10. Результаты преобразования

Задание. Проверьте, как преобразуются вводимые числа. Проверьте, что происходит при выходе за границы допустимых значений. Вычислите минимальное и максимальное значения с помощью **Калькулятора** — **Calculator**, переключив его в режим научных вычислений.

В наших опытах нам потребуется значение больше, чем два миллиарда. Для этого придётся использовать другой целый тип — **long long** (8 байт).

Немного изменим нашу программу (рис. 5.11).

Для преобразования строки символов в длинное целое тоже нужна другая функция с похожим названием:

long long atoll (const char *str).

Расшифровка названия:

Convert «a» (string of symbols) to long long integer.

Вывод на экран длинного целого тоже требует своего, особого формата:

%lld.

E:\Arkov-OMP\omp-17-atoll\omp-17-atoll.c

```c
#include <stdio.h>
#include <stdlib.h>
void main(int argc, char* argv[]) {
    long long N;
    N = atoll(argv[1]);
    printf("a = %s\n", argv[1]);
    printf("N = %lld\n", N);
}
```

Рис. 5.11. Преобразование строки в длинное целое

Запускаем нашу новую программу с разными входными значениями (рис. 5.12).

Здесь мы тоже можем найти пределы, выше которых подняться не удаётся.

Результаты нас устраивают. Нам пока что такое большое число не грозит.

```
E:\Arkov-OMP\omp-17-atoll\Release>omp-17-atoll 12
a = 12
N = 12

E:\Arkov-OMP\omp-17-atoll\Release>omp-17-atoll 100000000000
a = 100000000000
N = 100000000000

E:\Arkov-OMP\omp-17-atoll\Release>omp-17-atoll -100000000000
a = -100000000000
N = -100000000000

E:\Arkov-OMP\omp-17-atoll\Release>omp-17-atoll 10000000000000000000
a = 10000000000000000000
N = 9223372036854775807

E:\Arkov-OMP\omp-17-atoll\Release>omp-17-atoll -10000000000000000000
a = -10000000000000000000
N = -9223372036854775808
```

Рис. 5.12. Результаты преобразования

Задание. Составьте программу (рис. 5.11) и проверьте правильность её работы.

5.4. УПРАВЛЕНИЕ ЦИКЛОМ

Мы научились вводить число из командной строки. Теперь мы сможем управлять поведением программы, не изменяя исходный текст.

Возьмём нашу «большую задачу» параллельного суммирования из раздела 4 и немного её усложним (рис. 5.13).

Зададим число итераций цикла из командной строки (строка 11).

А программа пусть выводит на экран время расчётов (строка 25).

E:\Arkov-OMP\omp-18-atoll-mask\omp-18-atoll-mask.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>
#include <windows.h>
void main(int argc, char* argv[]) {
    long long N, i, S = 0;
    int TNum, Mask;
    clock_t t;
    double Twin, Tomp;
    N = atoll(argv[1]);
    t = clock();
    Tomp = omp_get_wtime();
#pragma omp parallel for reduction(+:S) private (TNum, Mask)
    for (i = 0; i < N; i++) {
        if (S == 0) {
            TNum = omp_get_thread_num();
            Mask = pow(2, TNum);
            SetThreadAffinityMask(GetCurrentThread(), Mask); }
        S++; }
    t = clock() - t;
    Tomp = omp_get_wtime() - Tomp;
    Twin = (float)t / CLOCKS_PER_SEC;
    printf("%lld\t%lld\t", N, S);
    printf("%2.10f\t%2.10f\n", Twin, Tomp); }
```

Рис. 5.13. Управление циклом из командной строки

Запускаем программу в командной строке и подбираем число итераций в цикле. Нам нужно получить приемлемое время выполнения программы — около 20 секунд. Затем запускаем программу несколько раз (рис. 5.14).

Тем временем следим за загрузкой ядер процессора. Должно быть 100%

```
E:\Arkov-OMP\omp-18-atoll-mask\Release>omp-18-atoll-mask 100000000000
100000000000    100000000000    26.6459999084   26.6460832832

E:\Arkov-OMP\omp-18-atoll-mask\Release>omp-18-atoll-mask 100000000000
100000000000    100000000000    26.0750007629   26.0753918584

E:\Arkov-OMP\omp-18-atoll-mask\Release>omp-18-atoll-mask 100000000000
100000000000    100000000000    25.8910007477   25.8915533145
```

Рис. 5.14. Четыре запуска

Задание. Составьте программу (рис. 5.13) и подберите приемлемое число итераций. Запустите программу несколько раз и обратите внимание на длительность решения задачи.

5.5. АНАЛИЗ ЭФФЕКТИВНОСТИ

У нас практически всё готово для анализа эффективности распараллеливания.

Напомним две основных характеристики распараллеливания:

— **speedup** — ускорение;

— **efficiency** — эффективность.

Формулы для вычисления этих показателей приводятся на рис. 5.15.

$$S_p = \frac{T_1}{T_p} \qquad E_p = \frac{S_p}{p}$$

Рис. 5.15. Характеристики распараллеливания

Обозначения в формулах:

T1 — время выполнения последовательной программы

(на одном процессоре);

p — количество виртуальных процессоров, работающих параллельно при выполнении программы;

Tp — время выполнения параллельной программы на p процессорах;

Sp — ускорение вычислений на p процессорах;

Ep — эффективность параллельных вычислений на p процессорах.

Показатель ускорения говорит о том, во сколько раз быстрее работает параллельная программа. Можно сказать, что ускорение измеряется «в разах».

Ускорение вычислений не надо путать с ускорением в физике. В разделе «Механика» ускорение измеряется в квадратных метрах в секунду и отражает скорость изменения скорости движения.

Показатель эффективности говорит о средней загрузке вычислительной мощности. Обычно измеряется в процентах. По сути, это эффективность использования (загрузки работой) доступных вычислительных элементов.

Задание. Изучите в английской версии Википедии статью «Speedup» и обратите внимание на определение терминов «ускорение» и «эффективность». Выясните, что такое «сверхлинейное ускорение» (Super-linear speedup) и за счёт чего оно достигается.

5.5.1. ПАКЕТНЫЙ ФАЙЛ

Создаём пакетный файл в каталоге **Release** текущего проекта (рис. 5.16).

В первой строчке пакетного файла мы отключаем вывод на экран выполняемых команд.

Во второй строчке мы формируем строку заголовков полей (столбцов), разделяя их символом табуляции.

Далее следуют несколько циклов. Запускаем нашу программу с разным количеством итераций по 10 раз.

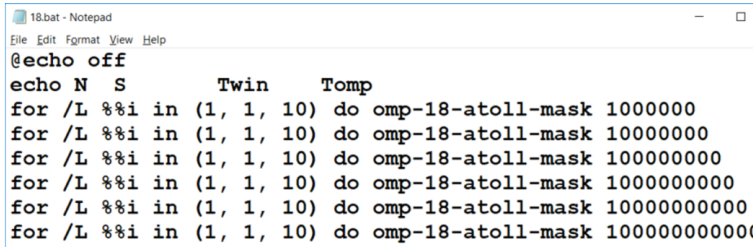


Рис. 5.16. Пакетный запуск

Запускаем пакетный файл и перенаправляем вывод в текстовый файл:

18> 18.txt.

Следим за загрузкой ядер (рис. 5.17). При каждом запуске загрузка поднимается до 100%. Это радует.

Попутно обратим внимание на дополнительные сведения в **Диспетчере задач**:

— один процессор **(Sockets — 1)**;

— четыре ядра **(Cores — 4)**;

— восемь логических (виртуальных) процессоров **(Logical processors — 8)**.

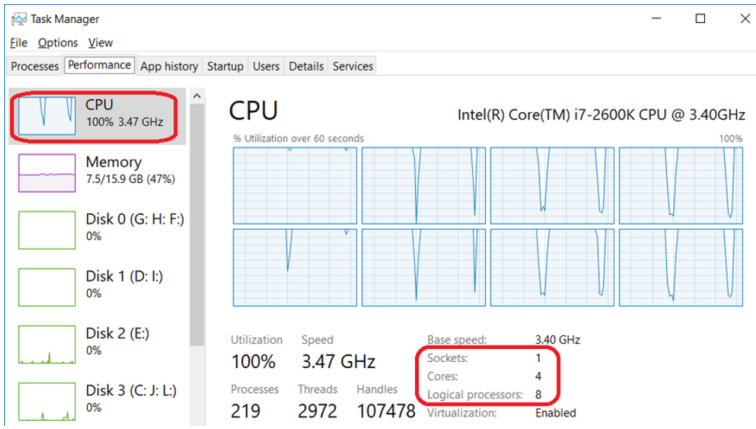


Рис. 5.17. Загрузка процессора

Задание. Запустите пакетный файл (рис. 5.16) и направьте вывод результатов в текстовый файл.

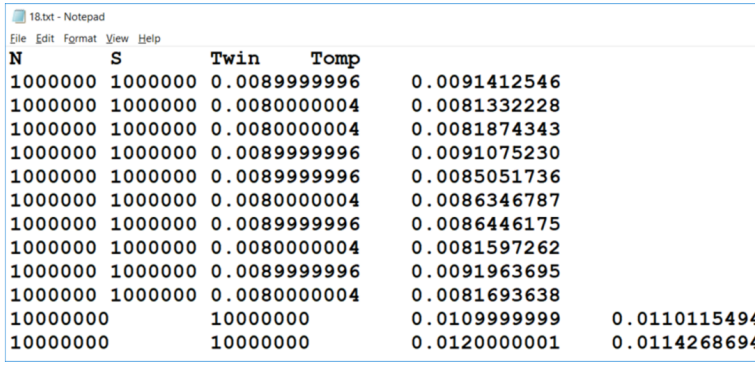
Получаем текстовый файл.

Открываем его в Блокноте:

Notepad.

Убеждаемся, что здесь записаны колонки цифр (рис. 5.18).

Когда число оказывается длиннее, чем восемь знаков, получаем смещение на следующую позицию табуляции. Надеемся, это не помешает импорту данных в Excel.


18.txt - Notepad

File Edit Format View Help

N	S	Twin	Tomp		
1000000	1000000	0.0089999996		0.0091412546	
1000000	1000000	0.0080000004		0.0081332228	
1000000	1000000	0.0080000004		0.0081874343	
1000000	1000000	0.0089999996		0.0091075230	
1000000	1000000	0.0089999996		0.0085051736	
1000000	1000000	0.0080000004		0.0086346787	
1000000	1000000	0.0089999996		0.0086446175	
1000000	1000000	0.0080000004		0.0081597262	
1000000	1000000	0.0089999996		0.0091963695	
1000000	1000000	0.0080000004		0.0081693638	
10000000		10000000		0.0109999999	0.0110115494
10000000		10000000		0.0120000001	0.0114268694

Рис. 5.18. Результаты работы

Задание. Изучите содержимое полученного текстового фйала в **Блокноте.**

5.5.2. СВОДНАЯ ТАБЛИЦА

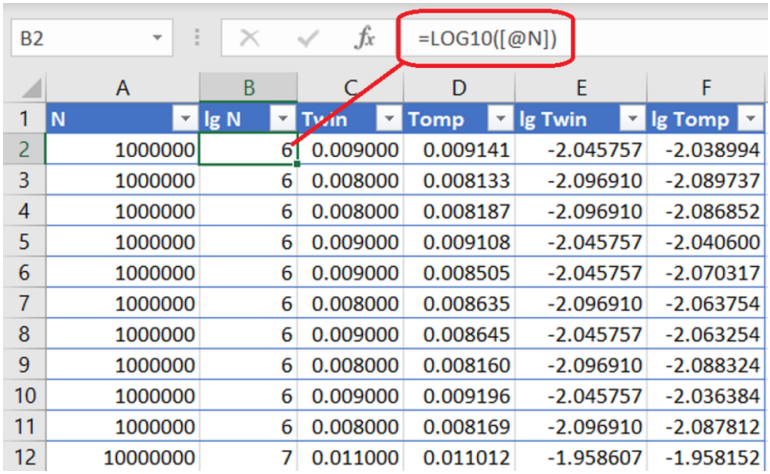
Загружаем текстовый файл в Excel. Указываем символ табуляции в качестве разделителя полей.

При необходимости указываем, что в роли разделителя целой и дробной частей у нас используется точка. Это особенно важно при загрузке данных в русскую версию пакета. Это делается на третьем шаге загрузки с помощью Мастера импорта текстовых файлов:

Text Import Wizard — Step 3 of 3 — Advanced — Decimal separator –.

После загрузки форматируем полученную таблицу.

Добавляем вычисление десятичного логарифма от числа итераций и времени выполнения — три дополнительных стобца (рис. 5.19).



	A	B	C	D	E	F
1	N	lg N	Twin	Tomp	lg Twin	lg Tomp
2	1000000	6	0.009000	0.009141	-2.045757	-2.038994
3	1000000	6	0.008000	0.008133	-2.096910	-2.089737
4	1000000	6	0.008000	0.008187	-2.096910	-2.086852
5	1000000	6	0.009000	0.009108	-2.045757	-2.040600
6	1000000	6	0.009000	0.008505	-2.045757	-2.070317
7	1000000	6	0.008000	0.008635	-2.096910	-2.063754
8	1000000	6	0.009000	0.008645	-2.045757	-2.063254
9	1000000	6	0.008000	0.008160	-2.096910	-2.088324
10	1000000	6	0.009000	0.009196	-2.045757	-2.036384
11	1000000	6	0.008000	0.008169	-2.096910	-2.087812
12	10000000	7	0.011000	0.011012	-1.958607	-1.958152

Рис. 5.19. Загруженная таблица

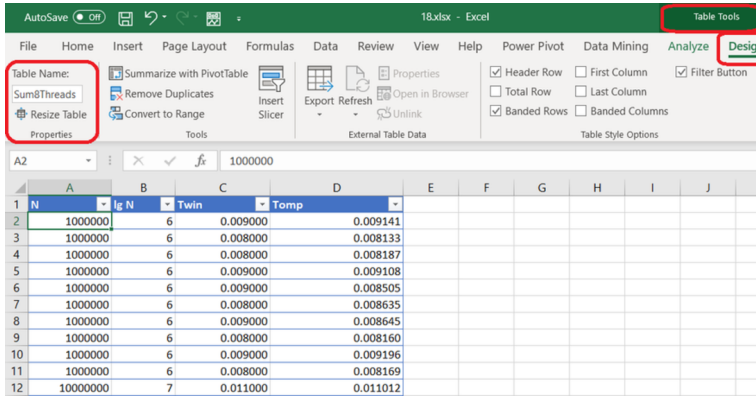
Задание. Загрузите таблицу и добавьте вычисление логарифма.

Данные загружена и подготовлены. Переходим к анализу результатов.

Щёлкаем по нашей таблице. Присвоим таблице уникальное имя:

Table Tools — Design — Properties — Table Name (рис. 5.20).

С помощью названия **Sum8Threads** мы напомним себе, что это была программа суммирования и в этом опыте были задействованы 8 параллельных потоков.



	A	B	C	D
1	N	lg N	Twin	Tomp
2	1000000	6	0.009000	0.009141
3	1000000	6	0.008000	0.008133
4	1000000	6	0.008000	0.008187
5	1000000	6	0.009000	0.009108
6	1000000	6	0.009000	0.008505
7	1000000	6	0.008000	0.008635
8	1000000	6	0.009000	0.008645
9	1000000	6	0.008000	0.008160
10	1000000	6	0.009000	0.009196
11	1000000	6	0.008000	0.008169
12	10000000	7	0.011000	0.011012

Рис. 5.20. Имя таблицы

Задание. Присвойте таблице короткое информативное имя.

Построим сводную таблицу (Pivot Table).

Выбираем в верхнем меню вставку сводной таблицы:

Insert — Tables — PivotTable.

Делаем настройки в диалоговом окне:

Create PivotTable.

Убеждаемся, что данные для анализа будут взяты из нашей таблицы:

Choose the data that you want to analyze — Select a table or range — Sum8Threads.

Указываем место для сводной таблицы на текущем листе:

Choose where you want the PivotTable report to be placed — Existing Worksheet.

В строчке **Location** выбираем ячейку на текущем листе. Это будет левый верхний угол нашей сводной таблицы (рис. 5.21).

Нажимаем **OK**.

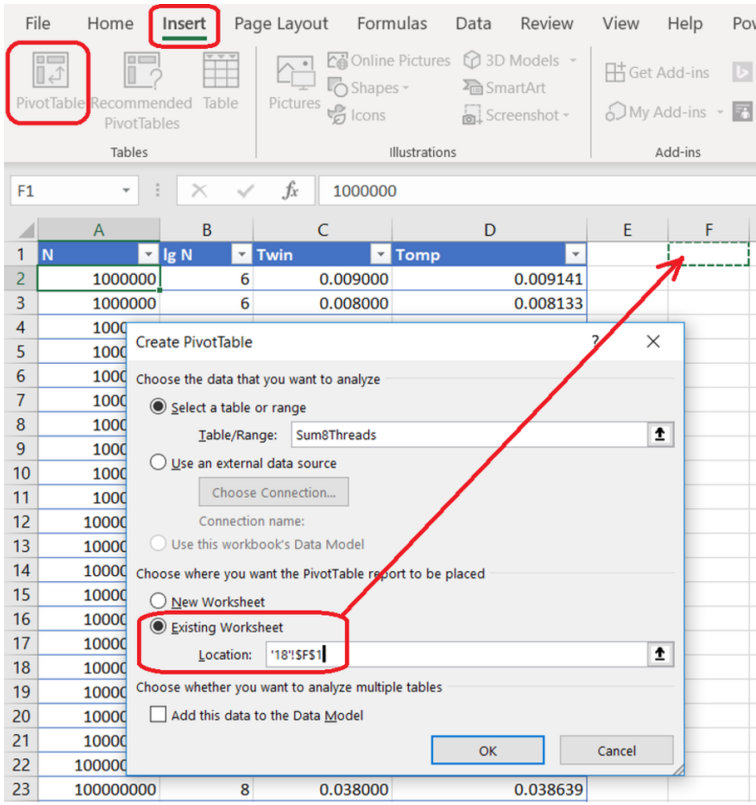
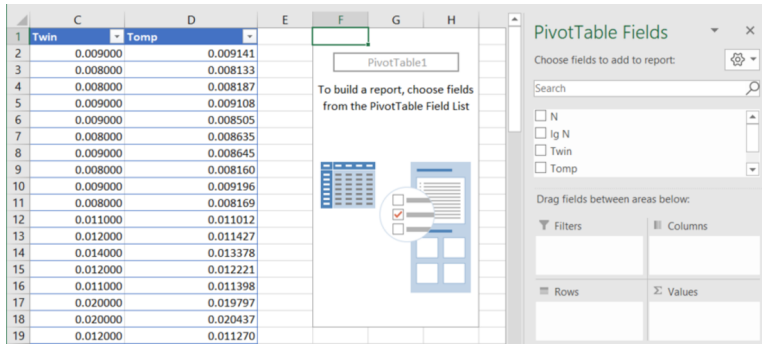


Рис. 5.21. Вставка сводной таблицы

На текущем листе появляются очертания будущей сводной таблицы (рис. 5.22).

В правой части окна выводится область настройки полей: **PivotTable Filelds.**



	C	D
1	Twin	Tomp
2	0.009000	0.009141
3	0.008000	0.008133
4	0.008000	0.008187
5	0.009000	0.009108
6	0.009000	0.008505
7	0.008000	0.008635
8	0.009000	0.008645
9	0.008000	0.008160
10	0.009000	0.009196
11	0.008000	0.008169
12	0.011000	0.011012
13	0.012000	0.011427
14	0.014000	0.013378
15	0.012000	0.012221
16	0.011000	0.011398
17	0.020000	0.019797
18	0.020000	0.020437
19	0.012000	0.011270

Рис. 5.22. Заготовка сводной таблицы

Настроим поля сводной таблицы (рис. 5.23).

Перетаскиваем поля из списка в области строк и столбцов:

Rows — lg N;

Values — lg Twin, lg Tomp.

После такой настройки в сводной таблице появляются числовые значения.

В разделе значений **Values** мы видим указание на вычисление суммы значений:

Sum of lg Twin;

Sum of lg Tomp.

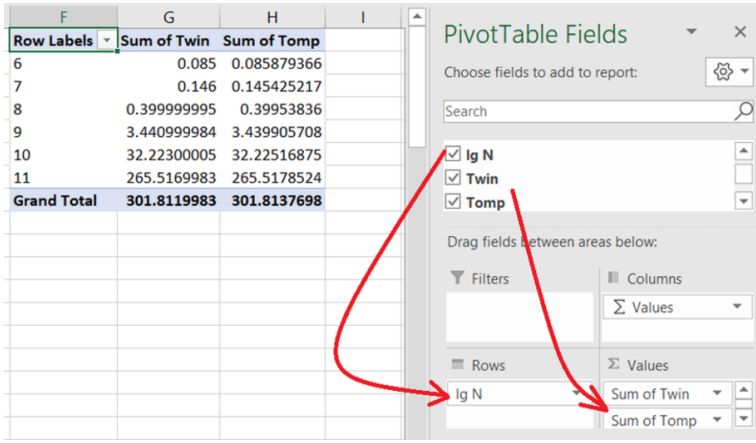


Рис. 5.23. Выбор полей

Настроим вычисление средних значений:

Щелкаем по полю **Sum of Tomp** в области значений **Values** и выбираем в выпадающем меню **Value Field Settings** (рис. 5.24).

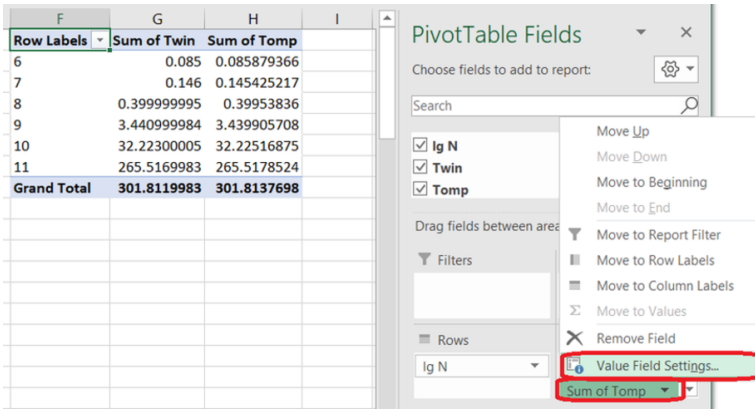


Рис. 5.24. Настройка значений

В диалоговом окне **Value Field Settings** выбираем вычисление среднего значения:

Summarize value field by — Average.

Настраиваем вычисления среднего для обоих отсчётов времени.

Убеждаемся, что в сводной таблице появились средние вместо сумм (рис. 5.25).

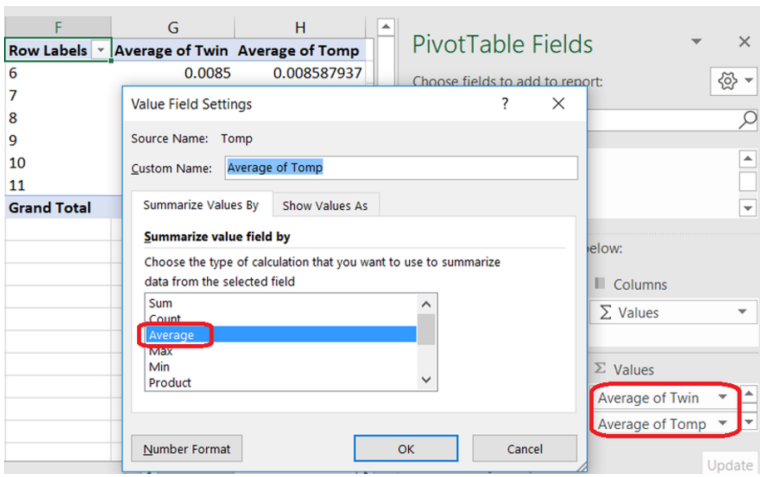


Рис. 5.25. Вычисление среднего

Задание. Настройте сводную таблицу, как описано выше.

Мы определили среднее время выполнения программы.

Сравним измерения времени, полученные двумя разными библиотечными функциями — **Windows** и **OpenMP**.

Найдём разность **(Twin — Tomp)** и выразим её в процентах от **Tomp** (рис. 5.26).

Поскольку мы ссылаемся на ячейки сводной таблицы, в формуле появляется вызов специальной функции

GETPIVOTDATA

При этом мы не можем копировать формулы. Нам придётся ввести формулы во все ячейки вручную.

=(GETPIVOTDATA("Average of Twin",H1,"lg N",6)-GETPIVOTDATA("Average of Tomp",H1,"lg N",6))/GETPIVOTDATA("Average of Tomp",H1,"lg N",6)

F	G	H	I	J	K	L	M	N	O
lg Tomp		Row Labels	Average of Twin	Average of Tomp	Average of lg Twin	Average of lg Tomp		Error %	
-2.038994		6	0.008500	0.008587937	-2.071334	-2.066603		-1.02395%	
-2.089737		7	0.014600	0.014542522	-1.850988	-1.853952		0.39524%	
-2.086852		8	0.040000	0.039953836	-1.398227	-1.398653		0.11554%	
-2.040600		9	0.344100	0.343990571	-0.463746	-0.463885		0.03181%	
-2.070317		10	3.222300	3.222516875	0.508139	0.508168		-0.00673%	
-2.063754		11	26.551700	26.55178524	1.423838	1.423840		-0.00032%	
-2.063254		**Grand Total**	**5.030200**	**5.030229497**	**-0.642053**	**-0.641847**			

Рис. 5.26. Оценка погрешности

Построим график в логарифмическом масштабе (рис. 5.27).

По оси «иксов» — логарифм числа итераций.

По оси «игреков» — логарифм времени выполнения.

Настраиваем масштаб по осям так, чтобы всё поле графика было занято нами данными.

Устанавливаем цвет маркеров и линий.

Можно видеть, что начиная с 10^8 начинается практически прямая линия.

Так что в дальнейших опытах можно ограничиться меньшим количеством вычислений.

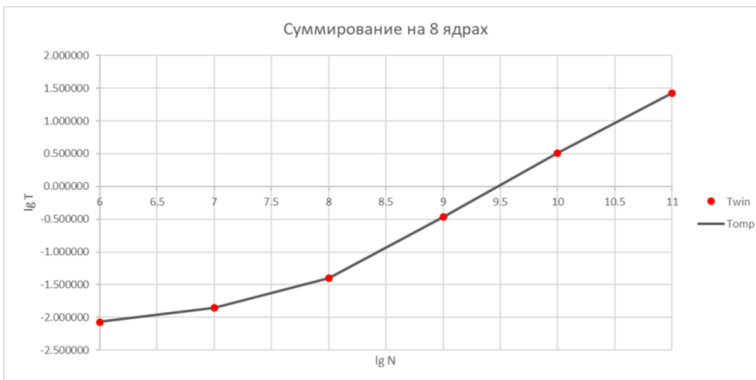


Рис. 5.27. Логарифмический масштаб

Задание. Сравните оценки времени выполнения, полученные разными способами.

5.5.3. ОРГАНИЗАЦИЯ ЭКСПЕРИМЕНТОВ

Нам предстоит провести несколько экспериментов. В этом разделе и в следующих разделах. И в следующих работах тоже. Мы много раз запустим одну и ту же программу. С разными настройками. На разном количестве ядер. Затем мы обработаем результаты и сделаем свои выводы.

Сейчас нас интересует ускорение параллельной программы.

Немного упростим предыдущий вариант параллельного суммирования. Пусть всё делается автоматически.

Текст программы приводится на рис.5.28.

В командной строке вводим количество параллельных потоков (строка 5).

Преобразуем введённую строку в целое число (строка 10).

Задаём число потоков в параметрах директивы parallel (строка 12).

Мы выбрали постоянное количество итераций в цикле. Пусть это будет совсем небольшое число. Всего несколько миллиардов (строка 13).

Определяем продолжительность расчётов (строки 11 и 15). Для используем библиотечную функцию **omp_get_wtime**. Мы уже убедились в точности её работы.

В конце программы выводим на экран число потоков, полученную сумму и время работы (строка 16).

Сумма должна быть всегда одна и та же. Это просто проверка правильности работы программы. На всякий случай. Если не выводить результат работы S на экран и никак его не использовать, компилятор может немного оптимизировать нашу программу. За ним нужен глаз да глаз.

E:\Arkov-OMP\omp-19-sum-bat\omp-19-sum-bat.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <windows.h>
void main(int argc, char* argv[])
{
    long long i, S = 0;
    double T;
    int p;
    p = atoi(argv[1]);
    T = omp_get_wtime();
#pragma omp parallel for reduction(+:S) num_threads(p)
    for (i = 0; i < 100000000000; i++)
        S++;
    T = omp_get_wtime() - T;
    printf("%d\t%lld\t%2.10f\n", p, S, T);
}
```

Рис.5.28. Параллельное суммирование

Задание. Составьте программу (рис. 5.28).

Программа готова.

Незабываем выбрать конфигурацию Release и включить поддержку OpenMP.

Скомпилируем её:

Build — Build Solution.

Исполняемый файл готов.

Создаём пакетный файл для многократного запуска с разным числом потоков (рис. 5.29).

Мы уже познакомились с использованием цикла в пакетных файлах.

Отключаем вывод команд на экран.

Выводим заголовок таблицы.

Далее вызываем по пять прогонов с числом потоков от одного до двенадцати.

Это решение задачи «в лоб».

Немного громоздко, но работоспособно.

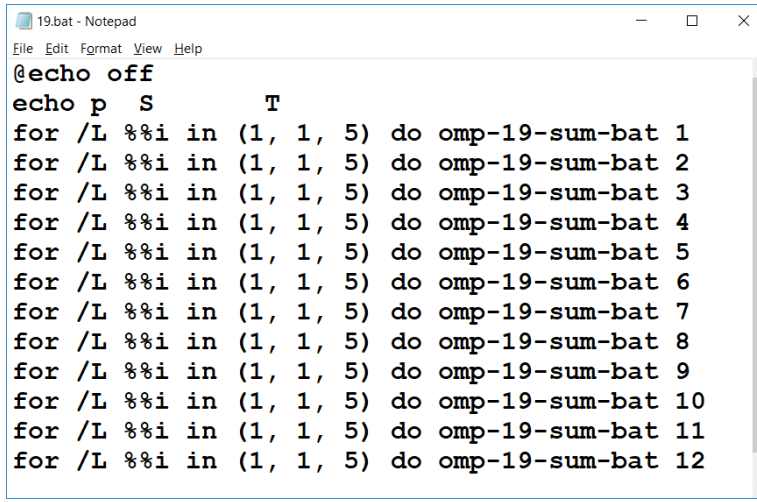


Рис. 5.29. Пакетный файл

Задание. Составьте пакетный файл (рис. 5.29).

На самом деле, мы можем организовать вложенные циклы. Ведь пакетный файл — это язык программирования. Почти как настоящий.

В результате у нас осталось всего три строчки в нашем пакетном файле (рис. 5.30). Такой файл легко отлаживать. Меньше шансов ошибиться.

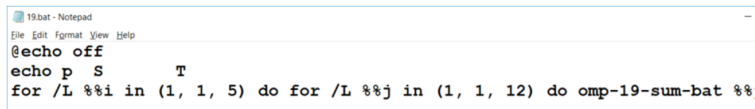


Рис. 5.30. Вложенные циклы

Задание. Составьте пакетный файл (рис. 5.30).

Переходим в командную строку.

Меняем текущий диск и каталог.

Запускаем наш пакетный файл и перенаправляем вывод в файл:

19> 19.txt.

Мы называем пакетный и текстовый файл одинаково – просто чтобы немного запутать читателя.

Загружаем полученный текстовый файл в Excel.

Обрабатываем данные с помощью сводной таблицы.

Строим график изменения времени выполнения от числа потоков (рис. 5.31).

Как ни странно, скорость работы почти не меняется, начиная с четырёх потоков.

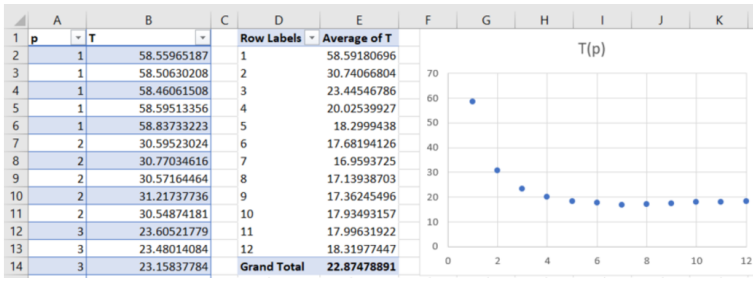


Рис.5.31.

Задание. Загрузите данные в Excel и постройте график (рис. 5.31).

Теперь наконец-то мы сможем найти показатели ускорения и эффективности.

У нас есть число потоков и время выполнения.

Будем считать, что число потоков равно числу логических процессоров. Мы притворимся, что запускали программу с **p** потоков на **p** процессорах.

Для дальнейших расчётов мы выделим сводную таблицу

и скопируем её в буфер обмена. Затем мы вставим её на свободном месте листа — но только как значения:

Paste Options — Values.

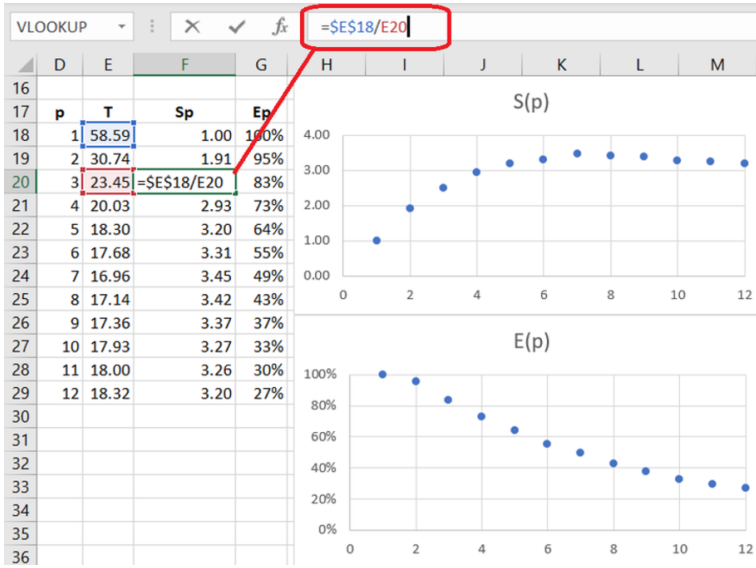
Добавим столбцы показателей $\mathbf{S_p}$ и $\mathbf{E_p}$ (рис. 5.32).

Мы сравниваем длительность работы параллельной программы с последовательной. Поэтому в формулах для расчёта ускорения мы фиксируем адрес первой ячейки **E18**, нажав клавишу **F4**. За счёт этого мы можем копировать нашу формулу в остальные ячейки столбца.

Строим графики ускорения и эффективности в зависимости от числа потоков (и ядер).

Рассмотрим график. Картина довольно неожиданная. Получается, что ускорение нашей программы не достигает 8 раз на восьми ядрах. Мы не достигли даже ускорения в четыре раза.

В нашей пробной программе мы работали с целыми числами. Складывали единички. Зато мы полностью отработали схему эксперимента и обработки данных. Это пригодится нам в следующем разделе.



p	T	Sp	Ep
1	58.59	1.00	100%
2	30.74	1.91	95%
3	23.45	=E18/E20	83%
4	20.03	2.93	73%
5	18.30	3.20	64%
6	17.68	3.31	55%
7	16.96	3.45	49%
8	17.14	3.42	43%
9	17.36	3.37	37%
10	17.93	3.27	33%
11	18.00	3.26	30%
12	18.32	3.20	27%

Рис. 5.32. Ускорение и эффективность

Задание. Постройте таблицу и графики (рис. 5.32). Сделайте выводы об ускорении программы.

Мы рассмотрели работу параллельной программы, которой потоки выполняются на любых ядрах. К тому же, операционная система норовит перебрасывать потоки с одного ядра на другое.

Интересно, что будет, если привязать каждый поток к своему ядру? Станет ли от этого программа работать быстрее?

Задание. Добавьте в программу параллельного суммирования привязку потоков к ядрам. Повторите исследование и выясните, как влияет привязка к ядрам на ускорение расчётов.

6. ЧИСЛЕННОЕ ИНТЕГРИРОВАНИЕ

Мы рассмотрели основы распараллеливания. Можем переходить к решению более реалистичной задачи.

Посмотрим, как покажет себя компьютер на задаче численного интегрирования. Там придётся работать с вещественными числами. А ведь операции с плавающей точкой выполняются гораздо дольше.

6.1. ВАРИАНТЫ ЗАДАНИЙ

В данной работе мы продолжаем рассматривать задачу численного интегрирования методом прямоугольников.

Формула приводится на рис. 6.1.

$$S = \int_{A}^{B} (a_3 x^3 + a_2 x^2 + a_1 x + a_0) dx$$

Рис. 6.1. Определённый интеграл

Мы используем те же самые варианты заданий, что и предыдущей работе (рис. 6.2).

Вариант	a_3	a_2	a_1	a_0	A	B
0	0,05	0,2	-10	200	-20	20
1	0,1	0,1	-10	200	-10	20
2	0,05	0,1	-10	100	-15	10
3	-0,1	0,5	-10	200	-20	5
4	0,05	0,3	-20	200	-5	20
5	0,03	0,3	-10	50	-20	10
6	0,02	0,2	-3	200	-20	5
7	0,05	0,1	-10	60	-15	5
8	0,04	0,3	-10	100	-5	20
9	0,05	0,5	-5	150	-20	20
10	0,06	0,3	-8	110	-10	20

Рис. 6.2. Варианты заданий

Номер варианта — последняя цифра номера зачётки. Для цифры ноль берём десятый вариант

Нам предстоит выбрать приемлемую продолжительность расчетов и определить ускорение и эффективность для различного числа потоков. Как и в предыдущих разделах, мы будем использовать в расчётах среднее время вычислений по нескольким запускам программы.

Задание. Опишите в отчёте свой вариант задания.

6.2. АНАЛИТИЧЕСКОЕ РЕШЕНИЕ

Мы будем определять значение определенного интеграла с помощью аналитических и численных методов.

В предыдущей работе [1] мы разработали и отладили про-

грамму. Теперь можно сосредоточиться на исследовании эффективности распараллеливания и не углубляться в тонкости программирования как такового.

В предыдущей работе мы уже выполнили следующие этапы:

— аналитическое решение задачи;
— последовательная программа;
— параллельные потоки.

В данной работе мы исследуем технологию автоматического создания параллельных потоков.

Точное аналитическое решение используется в дальнейшем для определения погрешности вычислений.

Задание. Запишите в отчёте аналитическое решение задачи — формулы и найденное значение.

6.3. ПОСЛЕДОВАТЕЛЬНАЯ ПРОГРАММА

Первый этап данной работы — последовательная программа численного интегрирования. Вычислим значение определенного интеграла, задавая разные значения числа прямоугольников и числа потоков.

6.3.1. ОДИНАРНАЯ ТОЧНОСТЬ

В этом разделе мы исследуем точность вычислений при использовании чисел одинарной точности.

Напомним, что тип **float** может обеспечить всего 7 десятичных разрядов для значащей части числа (мантиссы).

При численном интегрировании это приводит к потере точности. Когда значение накопленной суммы больше нового слагаемого в десять миллионов раз, значение практически полностью оказывается за пределами разрядной сетки. Новое число просто теряется, и сумма перестаёт расти. Таким образом, начиная с десяти миллионов итераций можно ожидать существенный рост ошибки.

Составим последовательную программу (рис. 6.3). Здесь мы используем приёмы, успешно отработанные в предыдущих разделах.

Вводим число итераций через параметр командной строки (строка 5).

Объявляем переменные одинарной точности типа **float** (строка 9).

Преобразуем вводимую строку символов в длинное целое (строка 10).

Время вычислений определим через библиотечную функцию OpenMP (строки 11 и 18).

Вычисляем шаг по оси «иксов» (строка 12).

На каждом шаге вычислений находим текущее значение аргумента функции (строка 14).

Прибавляем новое значение функции в середине очередного интервала по «икс» (строка 15).

После завершения цикла умножаем сумму значений функции на длину интервала, то есть на основание прямоугольника (строка 17).

В конце программы выводим на экран значения числа итераций, полученную оценку интеграла и время вычислений (строка 19). Значения разделяем символом табуляции. Так будет легче загрузить результаты в Excel.

E:\Arkov-OMP\omp-20-seq\omp-20-seq.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <windows.h>
void main(int argc, char* argv[])
{
    long long i, N;
    double T;
    float dx, x = 0.0, S = 0.0;
    N = atoll(argv[1]);
    T = omp_get_wtime();
    dx = 40.0 / N;
    for (i = 0; i < N; i++) {
        x = -20.0 + (i + 0.5) * dx;
        S = S + 0.05 * x * x * x + 0.2 * x * x - 10.0 * x + 200.0;
    }
    S = S * dx;
    T = omp_get_wtime() - T;
    printf("%lld\t%2.10f\t%2.10f\n", N, S, T);
}
```

Рис. 6.3. Последовательная программа

Задание. Составьте программу (рис. 6.3) для своего варианта задания. Проверьте работу программы в командной строке, задавая разное количество итераций.

Запустим программу через пакетный файл (рис. 6.4).

Мы используем почти логарифмическое приращение числа итераций. Это позволит нам подробно рассмотреть начало графика и быстрой пройтись по большим значениям.

На досуге можно попробовать сделать пакетный файл более компактным. При этом придётся учесть такие условия:

— количество итераций — целое число;

— шаг приращения счётчика цикла — постоянный;

— шаг приращения числа итераций растёт логарифмически.

Конечно, это будет задачей повышенной сложности.

Нам сейчас пока что нужно просто посмотреть на поведение

программы. Мы обойдёмся таким грубым решением, как шесть строчек, прописанных вручную.

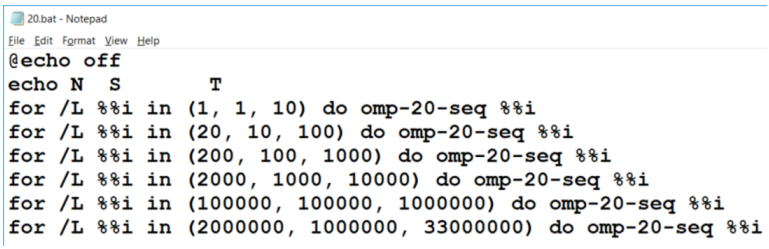


Рис. 6.4. Пакетный файл

Задание. Составьте пакетный файл (рис. 6.4) и запустите его в командной строке.

Запустим пакетный файл и перенаправим вывод в файл.

Загрузим файл в пакет Excel и проведём анализ полученных результатов.

Начнём с оценки интеграла.

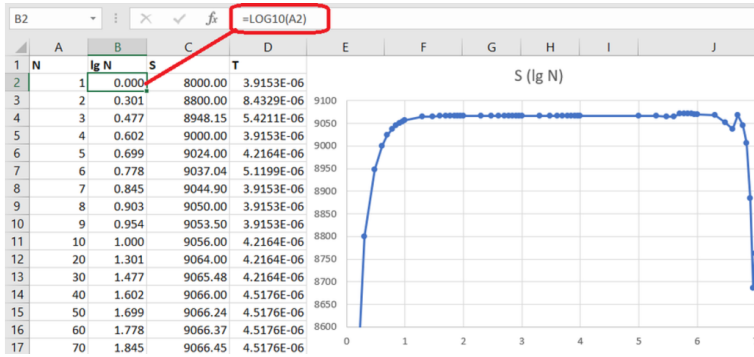
Построим график в логарифмическом масштабе. Находим десятичный логарифм числа итераций.

Рассмотрим полученный график (рис. 6.5).

Значение быстро растёт от 1 до 10 прямоугольников.

Затем значение стабилизируется.

Затем значение начинает падать после миллиона прямоугольников (то есть 10 в шестой степени). Это соответствует нашим прогнозам погрешности.



	A	B	C	D
1	N	lg N	S	T
2	1	0.000	8000.00	3.9153E-06
3	2	0.301	8800.00	8.4329E-06
4	3	0.477	8948.15	5.4211E-06
5	4	0.602	9000.00	3.9153E-06
6	5	0.699	9024.00	4.2164E-06
7	6	0.778	9037.04	5.1199E-06
8	7	0.845	9044.90	3.9153E-06
9	8	0.903	9050.00	3.9153E-06
10	9	0.954	9053.50	3.9153E-06
11	10	1.000	9056.00	4.2164E-06
12	20	1.301	9064.00	4.2164E-06
13	30	1.477	9065.48	4.2164E-06
14	40	1.602	9066.00	4.5176E-06
15	50	1.699	9066.24	4.5176E-06
16	60	1.778	9066.37	4.5176E-06
17	70	1.845	9066.45	4.5176E-06

Рис. 6.5. Значение интеграла

Задание. Запустите пакетный файл, загрузите данные в Excel и постройте график (рис. 6.5)

Продолжим наш анализ результатов многократных прогонов.

Построим график продолжительности вычислений в зависимости от числа итераций (прямоугольников). Зависимость практически линейная (рис. 6.6).

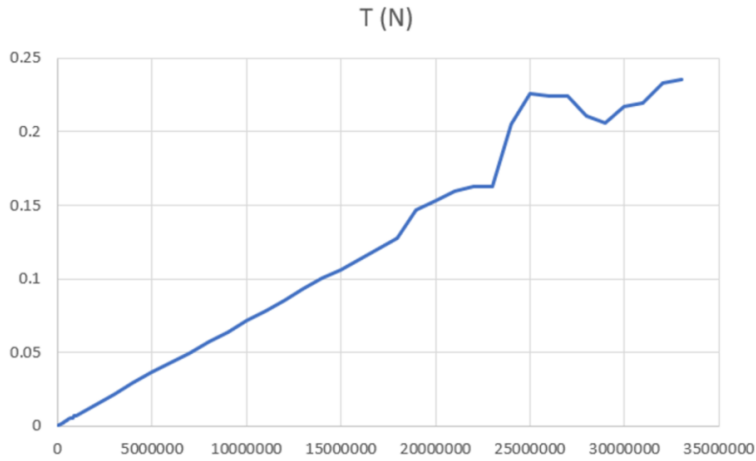


Рис. 6.6. Время вычислений

Задание. Постройте график (рис. 6.6) и опишите его в отчёте.

Теперь рассмотрим погрешность вычислений.

Сравним полученные оценки с аналитическим решением (рис. 6.7).

Построим график погрешности в зависимости от логарифма числа итераций (прямоугольников).

Можно видеть, что погрешность быстро падает практически до нуля при ста прямоугольниках и начинает возрастать после миллиона.

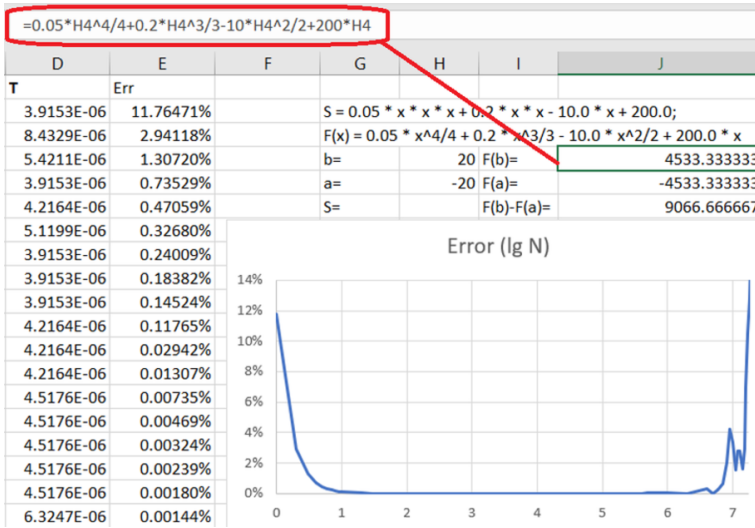


Рис. 6.7. Погрешность расчётов

Задание. Постройте график погрешности (рис. 6.7) и опишите его в отчёте.

Если внимательно рассмотреть таблицу значений погрешности, можно обнаружить ещё кое-что.

Особенно хорошо видна закономерность в логарифмическом масштабе. На таких графиках экспонента превращается в прямую линию.

Задание. Постройте график логарифма ошибки от логарифма числа итераций и найдите число итераций, начиная с которого начинает возрастать погрешность.

6.3.2. ДВОЙНАЯ ТОЧНОСТЬ

Повторим исследование погрешности вычислений.

Определим наши переменные как вещественные с двойной точностью **double**.

Повторим наше исследование и построим такие графики, как и в предыдущем разделе (рис. 6.8).

Нас будет интересовать поведение погрешности вычислений и подбор наилучших параметров интегрирования.

А ещё неплохо было бы сравнить результаты с предыдущим разделом. Повлияет ли разная точность представления переменных на качество оценок?

N	S	T	lg N	Err	Lg Err
1	8000	0.00000030	0.00	11.764706%	-0.929
2	8800	0.00000030	0.30	2.941176%	-1.531
4	9000	0.00000030	0.60	0.735294%	-2.134
10	9056	0.00000030	1.00	0.117647%	-2.929
22	9064.463	0.00000090	1.34	0.024307%	-3.614
47	9066.184	0.00000090	1.67	0.005326%	-4.274
100	9066.56	0.00000120	2.00	0.001176%	-4.929
220	9066.645	0.00000181	2.34	0.000243%	-5.614
470	9066.662	0.00000271	2.67	0.000053%	-6.274
100	9066.56	0.00000151	2.00	0.001176%	-4.929
220	9066.645	0.00000181	2.34	0.000243%	-5.614
470	9066.662	0.00000301	2.67	0.000053%	-6.274
100	9066.56	0.00000120	2.00	0.001176%	-4.929
220	9066.645	0.00000241	2.34	0.000243%	-5.614
470	9066.662	0.00000693	2.67	0.000053%	-6.274
1000	9066.666	0.00000572	3.00	0.000012%	-6.929

S= 9066.667

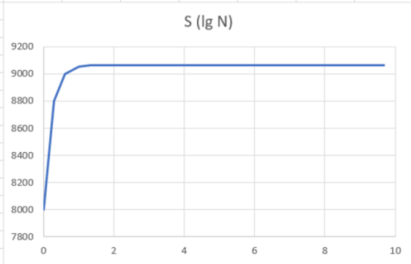


Рис. 6.8. Анализ результатов

Задание. Проведите анализ эффективности, постройте графики и сделайте выводы.

6.4. ПАРАЛЛЕЛЬНЫЕ ПОТОКИ

Составим параллельную программу (рис. 6.9).

Объявляем переменные одинарной точности (строка 10).

Вводим в командной строке число потоков **p** и число прямоугольников **N** (строки 11 и 12).

Выводим число потоков **p**, число прямоугольников **N**, оценку

интеграла **S** и время вычислений **T** (строка 22).

Компилируем программу, не забывая про поддержку OpenMP. Проверяем работоспособность в командной строке.

E:\Arkov-OMP\omp-22-par1\omp-22-par1.c

```
 1  #include <stdio.h>
 2  #include <stdlib.h>
 3  #include <omp.h>
 4  #include <windows.h>
 5  void main(int argc, char* argv[])
 6  {
 7      long long i, N;
 8      double T;
 9      int p;
10      float dx, x = 0.0, S = 0.0;
11      p = atoi(argv[1]);
12      N = atoll(argv[2]);
13      T = omp_get_wtime();
14      dx = 40.0 / N;
15  #pragma omp parallel for private(x) reduction(+:S) num_threads(p)
16      for (i = 0; i < N; i++) {
17          x = -20.0 + (i + 0.5) * dx;
18          S = S + 0.05 * x * x * x + 0.2 * x * x - 10.0 * x + 200.0;
19      }
20      S = S * dx;
21      T = omp_get_wtime() - T;
22      printf("%d\t%lld\t%2.10f\t%2.10f\n", p, N, S, T);
23  }
```

Рис. 6.9. Параллельная программа

Задание. Составьте параллельную программу (рис. 6.9) и проверьте её работоспособность.

Подбираем максимальное число прямоугольников так, чтобы выполнение программы занимало приемлемое время.

Составляем пакетный файл (рис. 6.10).

Здесь мы организуем три вложенных цикла:

— пять повторений для последующего усреднения;

— восемь значений числа потоков;

— число прямоугольников увеличиваем меняем логарифми-

чески.

22.bat - Notepad

File Edit Format View Help

```
@echo off
echo p  N        S        T
for /L %%x in (1,1,5) do for /L %%p in (1, 1, 8) do
for %%N in
(1,2,4,7,10,22,47,100,220,470,100,220,470,100,220,47
0,1000,2200,4700,10000,22000,47000,100000,220000,470
000,100000,220000,470000,1000000,2200000,4700000,100
00000,22000000,47000000,100000000,220000000,47000000
0,1000000000) do omp-22-par1 %%p %%N
```

Рис. 6.10. Пакетный файл

Задание. Составьте пакетный файл (рис. 6.10).

Запускаем пакетный файл и направляем вывод в файл.
Загружаем полученный текстовый файл в Excel.
Вставляем сводную таблицу (рис. 6.11):
— по строкам — число прямоугольников;
— по столбцам — число потоков.

Average of T	p							
N	1	2	3	4	5	6	7	8
1	0.000017	0.001641	0.002552	0.004124	0.004828	0.006224	0.007172	0.008947
2	0.000017	0.001626	0.002434	0.003502	0.004916	0.006161	0.007228	0.008356
4	0.000017	0.001734	0.003049	0.004138	0.005242	0.005830	0.007618	0.008427
7	0.000019	0.001399	0.002689	0.003934	0.005290	0.006289	0.007647	0.008490
10	0.000017	0.002081	0.002896	0.003732	0.005092	0.007416	0.007667	0.008888
22	0.000018	0.001496	0.002570	0.003588	0.005087	0.006006	0.007326	0.008468
47	0.000018	0.001732	0.002616	0.003626	0.004903	0.005899	0.007656	0.008924
100	0.000019	0.001756	0.002701	0.003911	0.004773	0.006418	0.008131	0.009676
220	0.000019	0.001745	0.002606	0.003783	0.005436	0.007219	0.007569	0.008574
470	0.000023	0.001855	0.003072	0.003666	0.005212	0.006081	0.007424	0.008614
1000	0.000028	0.005875	0.002660	0.003663	0.004890	0.006057	0.007082	0.008536
2200	0.000033	0.001793	0.002712	0.003732	0.005036	0.006414	0.007459	0.008785
4700	0.000051	0.001406	0.002636	0.003678	0.004792	0.005978	0.007660	0.008629
10000	0.000088	0.001580	0.003269	0.005382	0.004962	0.006149	0.007690	0.008817
22000	0.000178	0.001557	0.002616	0.003902	0.004843	0.006079	0.007138	0.010838

PivotTable Fiel..

Choose fields to add to report:

Search

p

Drag fields between areas below:

Filters | Columns: p

Rows: N | Values: Average of T

Рис. 6.11. Сводная таблица

Задание. Запустите пакетный файл, загрузите результаты

в Excel и постройте сводную таблицу (рис. 6.11).

Мы можем располагать данные в сводной таблице так. Как нам удобно. При этом все значения быстро пересчитываются автоматически.

Построим график, на котором можно будет увидеть, как меняется время вычислений **T** с увеличением числа прямоугольников **N**. Здесь же мы хотим отразить «расслоение» наших характеристик в зависимости от числа параллельных потоков (рис. 6.12). Включим легенду, чтобы видеть, какая ветка чему соответствует.

Можно видеть, что время работы программы для четырёх и восьми потоков практически не отличаются.

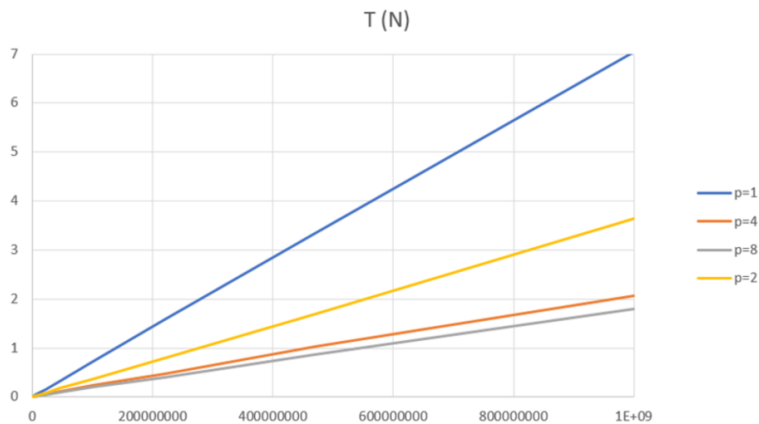


Рис. 6.12. Время вычислений

Задание. Постройте диаграмму времени вычислений (рис. 6.12).

Пришло время для анализа эффективности распараллеливания. Вставляем новую сводную таблицу. Проводим дополнительные вычисления.

Задание. Сформируйте новую сводную таблицу. Рассчитайте ускорение и эффективность. Постройте графики **lg Err (lg N)**, **Sp (lg N)** и **Ep (lg N)**. Сделайте выводы.

Мы подробно рассмотрели поведение параллельной программы с переменными одинарной точности.

В завершение работы повторим все этапы для вещественных переменных двойной точности. Сможем ли мы получить восьмикратное ускорение?

Задание. Проведите исследование распараллеливания с вещественными переменными типа **double**.

ЗАКЛЮЧЕНИЕ

В данной работе мы познакомились с технологией автоматического распараллеливания OpenMP для многоядерных процессоров. Мы убедились, что ускорение вычислений совсем не обязательно соответствует числу ядер или логических процессоров. Попробуйте повторить все наши опыты и самое главное — попробуйте объяснить полученные результаты.

ЛИТЕРАТУРА

1. Воеводин В. В., Воеводин Вл. В. Параллельные вычисления. — СПб.: БХВ-Петербург, 2002. — 608 с.

2. Гергель В. П. Высокопроизводительные вычисления для многопроцессорных многоядерных систем. — М.: МГУ, 2010. — 544 с.

3. Основы суперкомпьютерных технологий / Р.К.Газизов, В.О.Лукащук, С.Ю.Лукащук, А.В.Юлдашев. — Уфа: УГАТУ, 2008. — 267 с.

4. Немнюгин С. А., Стесик О. Д. Параллельное программирование для многопроцессорных вычислительных систем. — СПб.: БХВ-Петербург, 2002. — 400 с.

5. Эндрюс Г. Р. Основы многопоточного, параллельного и распределенного программирования. — М.: Вильямс, 2003.– 512 с.

6. Воеводин В. В. Вычислительная математика и структура алгоритмов. — М.: МГУ, 2006. — 112 с.

Режим доступа:

http://parallel.ru/info/parallel/voevodin/

7. Воеводин Вл. В., Жуматий С. А. Вычислительное дело и кластерные системы. — М.: МГУ, 2007. — 150 с.

Режим доступа:

http://parallel.ru/info/parallel/cluster/

8. Антонов А. С. Параллельное программирование с использованием технологии OpenMP: Учебное пособие. — М.: МГУ, 2009. — 77 с.

Режим доступа:

http://parallel.ru/info/parallel/openmp/

9. Антонов А. С. Технологии параллельного программирования MPI и OpenMP. — М.: МГУ, 2012. — 344 с.

Режим доступа к примерам программ:

https://parallel.ru/tech/tech_dev/MPI&OpenMP/examples/

10. Арьков В. Ю. Организация параллельных потоков. Часть 1: Лабораторный практикум. — [б. м.]: Издательские решения, 2020. — 46 с.

Режим доступа:

https://ridero.ru/books/parallelnye_potoki_windows/

11. Арьков В. Ю. Бизнес-аналитика. Сводные таблицы. Часть 1: Учебное пособие. — [б. м.]: Издательские решения, 2020. — 180 с.

Режим доступа:

https://ridero.ru/books/biznes-analitika_v_excel_svodnye_tablicy/

Валентин Юльевич Арьков

Организация параллельных потоков. Часть 2

Учебное пособие

Создано в интеллектуальной издательской системе Ridero

Арьков Валентин Юльевич — доктор технических наук, профессор кафедры автоматизированных систем управления Уфимского государственного авиационного технического университета. Опыт преподавания в вузе — более 30 лет.

При решении инженерных, экономических и научных задач используются высокопроизводительные вычисления — High Performance Computing или сокращённо HPC.
Параллельные программы нужны для того, чтобы использовать вычислительные мощности многоядерных процессоров и графических ускорителей. В данной работе мы рассмотрим технологию автоматической организации параллельных потоков для многоядерных вычислительных машин.

ISBN 978-5-4498-5445-2



Rideró

Rideró.ru — издай книгу бесплатно!